

Shorter VOLE-in-the-Head-based Signatures from Vector Semi-Commitment

Seongkwang Kim¹, Byeonghak Lee¹, and Mincheol Son²

¹ Samsung SDS, Seoul, Korea
sk39.kim@samsung.com, byghak.lee@samsung.com

² KAIST, Daejeon, Korea
encrypted.def@kaist.ac.kr

Abstract. The VOLE-in-the-Head (VOLEitH) paradigm transforms VOLE-based zero-knowledge proofs into post-quantum signature schemes by allowing public verification. We introduce reduced VOLE-in-the-Head (rVOLEitH), which incorporates the Vector Semi-Commitment (VSC) technique. VSC, originally developed for MPC-in-the-Head (MPCitH) schemes, reduces commitment size while maintaining security by relaxing the binding property. We adapt the ideal cipher version of VSC (IC-VSC) into the VOLEitH framework, leading to a reduction in signature size. Our security analysis proves that rVOLEitH achieves existential unforgeability under chosen-message attacks (EUF-CMA) in the ideal cipher model. Compared to existing VOLEitH-based signatures, our approach reduces signature size by up to 6.0% while improving computational efficiency.

Furthermore, we analyze the impact of eliminating individual seed commitments and demonstrate a practical attack against a recently proposed VOLEitH variant that lacks such commitments. Our results establish rVOLEitH as an optimized and secure alternative for post-quantum signatures, improving both efficiency and security in the VOLEitH paradigm.

Keywords: VOLE-in-the-Head, Vector Semi-Commitment, Ideal Cipher Model, Zero-Knowledge Proofs, Post-Quantum Signatures

1 Introduction

Vector oblivious linear evaluation (VOLE) is a two-party cryptographic protocol that generalizes oblivious transfer by enabling one party (the sender) to hold a secret vector $\mathbf{q} \in \mathbb{F}^n$ and a scalar multiplier $\Delta \in \mathbb{F}$, while the other party (the receiver) selects an input vector $\mathbf{u} \in \mathbb{F}^n$ and learns the evaluation

$$\mathbf{v} = \Delta \cdot \mathbf{u} - \mathbf{q}, \tag{1}$$

without learning any additional information about $\mathbf{q}$ or Δ . Meanwhile, the sender remains oblivious to $\mathbf{u}$. This linear relationship enables efficient generation of correlated randomness and serves as a building block for various secure computation protocols [20, 7].

One of the applications is VOLE-based zero-knowledge (VOLE-ZK) protocols, which employs the VOLE primitive to design prover-efficient and linear-sized zero-knowledge proofs [23,24,3]. In these protocols, the prover and verifier preprocess random VOLE correlations to commit to all wire values in the circuit and subsequently prove multiplicative relations among these values using information-theoretic authentication techniques. However, a critical limitation of these protocols is that the verifier must be designated before the proof, as Δ in VOLE must remain secret and is chosen during the preprocessing stage.

To overcome this limitation, Baum et al. introduced VOLE-in-the-Head (VOLEitH) paradigm [5], a transformation of VOLE-ZK into a public-verifier proof using SoftSpoken OT [21]. This approach allows the verifier to delay choosing Δ in (1), thereby enabling public verifiability. Unlike prior LPN-based VOLE protocols, SoftSpoken OT relies only on Minicrypt assumptions, so a signature scheme based on the VOLEitH paradigm achieves security without relying on any mathematical hardness assumption in the random oracle model, assuming only the existence of a secure one-way function.

A predecessor to VOLEitH, MPC-in-the-Head (MPCitH) paradigm [17], follows a different approach in which the prover simulates a multi-party computation in their head. While MPCitH and VOLEitH differ conceptually, they share similar cryptographic techniques. Both rely on vector commitment (VC), which consists of Goldreich-Goldwasser-Micali (GGM) tree [13] to generate seeds, and expand them into random tapes using a pseudorandom generator (PRG).

In the context of MPCitH, Kim et al. introduced Vector Semi-Commitment (VSC), a more compact variant of vector commitment (VC). Unlike standard VC, VSC relaxes the binding property while still preventing signature forgery. An ideal cipher version of VSC proposed in [19] replaces almost all the random oracle calls in VC by ideal cipher calls to achieve computational efficiency, as well as enjoys the smaller signature size. The authors also proved the security of VSC-applied signature scheme in the ideal cipher model.

1.1 Our Contribution

In this paper, we investigate security and efficiency of VSC-applied VOLEitH-based signature scheme, dubbed *reduced VOLE-in-the-Head (rVOLEitH)*. We adopt all the techniques of the ideal cipher version of VSC (IC-VSC) proposed in [19], including commitment based on an ideal cipher, a correlated GGM tree, and extensive use of a fixed-key block cipher. We prove the EUF-CMA security of rVOLEitH in the ideal cipher model, assuming that the underlying one-way function is secure.

To prove the security of a scheme using VSC, the problematic thing is that VSC does not have the binding property. In original VOLEitH, since VOLECommit (VC and VOLE construction from VC) is extractable binding, an extractor can extract a VOLE pair from a forgery, and recover the witness from it. However, in the case of VSC, a naive extractor cannot extract a unique VOLE pair consistent with the leaf commitments. Therefore, we combine VSC and the consistency

check, called **ChosenVOLECommit**, and proved that it is extractable binding. The rest of the proof is similar since the witness can be recovered.

We evaluate the impact of VSC on signing/verification time and signature size by comparing rVOLEitH to existing VOLEitH-based signature schemes. Our reduced variants have reduced signature size by up to 6.0%, as detailed in Table 1. We estimated the signing/verification speed of rFAESTER – the rVOLEitH-variant for FAESTER [2] – using similar implementation and compare with FAESTER. It turns out that rFAESTER enjoys $2.2\times$ – $7.4\times$ faster speed than FAESTER.

What if the individual commitment to the seeds are not reduced but removed? This is a quite natural question from our work. We explore this by analyzing the recently proposed VOLEitH-based signature scheme from [16]. We will call the signature scheme *VOLEitH-SBC*. The security of this scheme relies on the subfield bilinear collision (SBC) problem, introduced by Huth and Joux [15]. The same work also proposes an MPCitH-based signature scheme where the one-way function is based on SBC. However, the VOLEitH variant deviates from the original structure of VOLEitH by omitting individual seed commitments. We demonstrate a practical attack on this scheme, requiring 2^{69} operations, exploiting this omission.

1.2 Related Work

VOLE-in-the-Head [5] is a new paradigm to build a post-quantum signature scheme relying on trapdoor-less primitives, such as one-way function [25,12,18], unstructured multivariate quadratic problem [8], or unstructured syndrome decoding problem [11]. Compared to MPC-in-the-Head [17] paradigm, which shares similar properties, VOLEitH-based signature schemes offer improved efficiency.

To efficiently generate randomness, both VOLEitH and MPCitH employ a GGM tree structure. Since computing the GGM tree constitutes a substantial portion of the signature size and signing/verification time, there have been many prior works on optimize its performance. Cui et al. applied the correlated GGM tree [14] to the VOLEitH-based signature scheme ReSolveD, optimizing the computation of GGM-based random vector commitments [11]. Huth and Joux utilized the sum-preserving property of correlated GGM trees to reduce signature size by eliminating the correction term for the secret key share from the signature [15]. In parallel, Bui et al. replaced the random oracle calls in the correlated GGM with efficient primitives, such as fixed-key AES, and integrated this optimization into VOLEitH-based signature schemes [10]. However, the provable security of their proposal did not exceed the birthday bound. Independently of the correlated GGM tree, Bui et al. proposed a fast, salted GGM tree that achieves full security in the ideal cipher model [9]. This salted tree maintains the evaluation speed of the best unsalted version while preventing multi-target attacks.

Finally, Kim et al. introduced a vector semi-commitment that relaxes the binding property of traditional vector commitments and applied it to a MPCitH-based signature scheme [19]. The vector semi-commitment allows an adversary

to find multiple preimages for a given commitment. However, this relaxation of the binding property does not directly lead to a signature forgery. Their vector semi-commitment scheme is instantiated in both the random oracle model and the ideal cipher model. When combined with recent GGM tree optimizations [10,9,15], their approach achieves fast, salted, half number of calls, reduced signature size, and the full security with respect to the number of queries to the ideal primitives.

2 Preliminaries

2.1 Notation

In this paper, we basically follow the notations from three papers [19,4,16]. In Section 3, we follow the notations from [19] but modified them to fit into FAEST notations [4]. Section 4 and 5 follows the notations of FAEST [4]. Section 6 follows the notations of [16]. For notations to be integrated, we starts array indices with 0, not 1, while arrays in [19] usually starts with index 1.

We denote $[n] = \{0, 1, \dots, n-1\}$ and $[n]^* = \{1, 2, \dots, n-1\}$. If this notation is used in an array representation such as $(a_i)_{i \in [n]}$, then this means an array with increasing index order $(a_0, \dots, a_{n-1})$. For integers $a < b$, We also denotes $[a, b] = \{a, a+1, \dots, b-1\}$. For a positive integer k and $0 \leq a < 2^{8k}$, $\langle a \rangle_{kB}$ is a canonical binary representation of a , which is an k -byte string. For a positive integer n and $k < n$, the falling factorial is denoted by $(n)_k = n \cdot (n-1) \cdot \dots \cdot (n-k+1)$. Unless stated otherwise, all logarithms are to the base 2.

For two vectors or strings a and b , their concatenation is denoted by $a||b$. For a vector $\mathbf{x}$, $\mathbf{x}[a : b]$ denotes a slice of the vector $\mathbf{x}$ from index a (inclusive) to b (exclusive). $\mathbf{x}[: b]$ and $\mathbf{x}[a :]$ denotes the same as $\mathbf{x}[0 : b]$ and $\mathbf{x}[a : \ell]$ respectively, where ℓ refers to the length of the vector $\mathbf{x}$. For a matrix $\mathbf{A}$, $\mathbf{A}[a : b]$ denotes a submatrix consisting of columns from index a (inclusive) to b (exclusive), and $\mathbf{A}|_{[a:b]}$ denotes a submatrix consisting of rows from index a (inclusive) to b (exclusive). For integers a and b , we denote the bitwise XOR of a and b by $a \oplus b$.

For a set S , we write $a \leftarrow_{\$} S$ to denote that a is chosen uniformly at random from S . Throughout this paper, the security parameter is denoted by λ .

2.2 VOLE-in-the-Head

VOLE-in-the-Head [5] is a technique that extends VOLE-ZK to achieve public verifiability. Except the random challenges between the steps, this technique consists of following 4 steps:

1. Random VOLE commitment,
2. VOLE consistency check,
3. VOLE-ZK proof,
4. VOLE key selection.

As the details of each step are key to reading this paper, we will explain each step in the following. As we focus on the signature scheme based on the VOLE-in-the-Head paradigm, we will concentrate on the non-interactive version of VOLE-in-the-Head. We basically follows the notation of [4].

Random VOLE Commitment. The prover simulates random VOLE protocol, rather than really invokes a VOLE protocol with the verifier. By using the Soft-Spoken VOLE protocol [21], the prover generates τ small-field VOLE correlations $(\mathbf{u}_i, \mathbf{v}_i)$ from GGM trees such that $\mathbf{q}_i = \Delta_i \cdot \mathbf{u}_i - \mathbf{v}_i$ for $i \in [\tau]$. Specifically, from each of the 2^k leaf nodes, vectors $\mathbf{r}_{i,j}$ are derived using a pseudorandom generator (PRG) and the prover gets

$$\mathbf{u}_i = \sum_{j \in \mathbb{F}_{2^k}} \mathbf{r}_{i,j}, \quad \mathbf{v}_i = \sum_{j \in \mathbb{F}_{2^k}} i \cdot \mathbf{r}_{i,j}.$$

Once the verifier learns all of the seeds except seed_{j^*} for some index j^* , it can compute

$$\mathbf{q}_i = \sum_{j \in \mathbb{F}_{2^k}} (\Delta_i - j) \cdot \mathbf{r}_i = \Delta_i \cdot \sum_{j \in \mathbb{F}_{2^k}} \mathbf{r}_{i,j} - \sum_{j \in \mathbb{F}_{2^k}} j \cdot \mathbf{r}_{i,j} = \Delta_i \cdot \mathbf{u}_i - \mathbf{v}_i,$$

where the verifier will know $(\Delta_i, \mathbf{q}_i)$. The small-field VOLE correlations are concatenated to a large field VOLE relation. Since each $\mathbf{u}_i$ are randomly generated, the prover generates the corrections $\mathbf{c}_i = \mathbf{u}_0 - \mathbf{u}_i$ for $i = 1, \dots, \tau - 1$. Then, the prover can get $\mathbf{u}$ and $\mathbf{v}$ such that $\mathbf{Q} = \Delta \cdot \mathbf{u} - \mathbf{V}$ where $\mathbf{Q} = [\mathbf{q}_0 \parallel \dots \parallel \mathbf{q}_{\tau-1}]$, $\mathbf{V} = [\mathbf{v}_0 \parallel \dots \parallel \mathbf{v}_{\tau-1}]$, and $\Delta = [\Delta_0 \parallel \dots \parallel \Delta_{\tau-1}]$ while Δ and $\mathbf{Q}$ are not chosen yet. To prevent a manipulation of the GGM trees, the prover commits to the leaf nodes, which will be called *leaf commitment*. Although the leaf commitment does not work as a commitment scheme when we deal with vector semi-commitment, we will stick with the same terminology if the context is clear.

This sequence of algorithms is called VOLE commitment (VOLECommit). It generates random VOLE correlations while delaying to choose the sender's part and commits to the leaf nodes. The VSC version of VOLECommit can be found in Algorithm 1. The corresponding verifier's function is called VOLE reconstruction (VOLEReconstruct). This function will be explained in the "VOLE key selection" part.

VOLE Consistency Check. As aforementioned, to concatenate the small-field VOLE relations, they should be consistent (i.e., $\mathbf{u}_i + \mathbf{c}_i = \mathbf{u}_0$ for $i = 1, \dots, \tau - 1$). The consistency is checked using a universal hash function (VOLEHash). For a linear universal hash function H , if $\mathbf{Q} = \Delta \cdot \mathbf{u} - \mathbf{V}$ is a valid VOLE correlation, then it should satisfies that $H(\mathbf{Q}) = \Delta \cdot H(\mathbf{u}) - H(\mathbf{V})$ with high probability, where H is applied in the column-major order. By putting $\tilde{\mathbf{u}} = H(\mathbf{u})$ into the signature, the verifier can check the VOLE consistency.

After the VOLE consistency check, the prover generates the commitment $\mathbf{d} = \mathbf{w} - \mathbf{u}$ to the witness $\mathbf{w}$. By using this commitment, the verifier can update the random VOLE correlations to chosen VOLE correlations by

$$\mathbf{Q} \leftarrow \mathbf{Q} + \Delta \cdot \mathbf{d} = \Delta \cdot (\mathbf{u} + \mathbf{d}) - \mathbf{V} = \Delta \cdot \mathbf{w} - \mathbf{V}.$$

VOLE-ZK Proof. To prove the knowledge of preimage of a one-way function, QuickSilver protocol [24] is used. The QuickSilver protocol [24] is an interactive constant-round zero-knowledge proof system which has been used in the VOLE-in-the-Head paradigm. To prove that a Boolean or arithmetic circuit C of t multiplication gates satisfies $C(\mathbf{w}) = 1$ for a witness $\mathbf{w} \in \mathbb{F}_2^n$, the prover and verifier invoke the standard subfield VOLE functionality $n + t$ times – once for each of the n input wires and the t multiplication-gate output wires – while addition gates incur zero communication. Concretely, for each such wire value w_i , the prover has a chosen VOLE correlation pair $(w_i, v_i) \in \mathbb{F}_2 \times \mathbb{F}_{2^k}$.

To verify the correctness of a multiplication gate indexed by (α, β, γ) , the verifier computes a value b_i as follows.

$$b_i = q_\alpha \cdot q_\beta - q_\gamma \cdot \Delta,$$

which expands to

$$b_i = v_\alpha \cdot v_\beta + (w_\alpha \cdot v_\beta + w_\beta \cdot v_\alpha - v_\gamma) \cdot \Delta + (w_\alpha \cdot w_\beta - w_\gamma) \cdot \Delta^2. \quad (2)$$

Here, b_i is linear in Δ if the prover computes w_γ correctly. To perform the verification, the prover provides $v_\alpha \cdot v_\beta$ and $(w_\alpha \cdot v_\beta + w_\beta \cdot v_\alpha - v_\gamma)$ to the verifier in a universal-hashed form (ZKHash). The verifier then substitutes these values into the Equation 2 and verifies that it holds.

The VOLE-ZK function in the FAEST document [4] is called AESProve or EMProve since the FAEST signature scheme proves the key knowledge of AES or Even-Mansour version of AES or Rijndael. In our paper, since the choice of one-way function does not matter, we describe the general functionality, called OWFProve, in Algorithm 12. The counterpart of OWFProve is called OWFVerify, and it is also described in Algorithm 13.

VOLE Key Selection. After all the commitments and proof are done, the prover randomly selects the VOLE key Δ by hashing the transcript. From chosen Δ , a hidden seed in each GGM tree is selected. To send all seeds except the single hidden seed per GGM tree, the prover puts partial decommitment $\mathbf{pdecom}_i$ which contains the cpath and leaf commitment to the hidden seed.

From a given $\mathbf{pdecom}_i$, the verifier can reconstruct the random VOLE correlation $(\mathbf{Q}, \Delta)$ by using VOLEReconstruct. By computing $\mathbf{Q} \leftarrow \mathbf{Q} + \Delta \cdot \mathbf{d}$, the verifier can also reconstruct the chosen VOLE correlations. The VSC version of VOLEReconstruct can be found in Algorithm 2.

Universality of VOLEHash and ZKHash. Let $\delta > 0$, and let $H : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ be a keyed function for three non-empty sets $\mathcal{K}$, $\mathcal{X}$, and $\mathcal{Y}$. A linear function H is said to be δ -almost universal (AXU) if for any $X \in \mathcal{X}$ and $Y \in \mathcal{Y}$,

$$\Pr[K \leftarrow_{\S} \mathcal{K} : H_K(X) = Y] \leq \delta.$$

VOLEHash (Algorithm 8) and ZKHash (Algorithm 9) are almost universal hash functions, and the almost universality of VOLEHash and ZKHash is shown in [4]; we will denote the almost universality of VOLEHash and ZKHash by ε_v and ε_{zk} , respectively.

3 Vector Semi-commitment

Vector semi-commitment (VSC) was proposed to relax the concept of vector commitment (VC) in the context of MPCitH-based signature schemes [19]. A VSC scheme, similarly as VC schemes, abstracts the process of generating views and corresponding commitments. While VSC does not provide full binding property, it reduces signature size without security degradation for some parameter choices. In the context of VOLEitH-based signature schemes, the definition and interface of VSC is same with VC but meaning of each operation is slightly different.

Although Kim et al. [19] proposed VSC schemes based on both random oracles (RO-VSC) and an ideal cipher (IC-VSC), we will stick to the ideal cipher version for better performance. So we will describe VSC in the ideal cipher model throughout this paper. We believe that the security of the random oracle version (with VOLEitH) will be more easy to proved, and most of the proof will be an analogue of the security proof of the ideal cipher version.

A VSC scheme consists of four algorithms: **Commit**, **Open**, **Recon**, and **Verify**. The **Commit** algorithm generates the VOLE shares and commits to the seeds of the VOLE shares. When a verifier challenges which shares to reveal, the prover uses **Open** to disclose a subset of the shares. **Open** produces partial decommitment information. Using this partial decommitment information, the verifier runs **Recon** to reconstruct the revealed shares and the corresponding commitments. Finally, the verifier uses **Verify** to check the validity of the partial decommitment information. We briefly introduce the interface of vector semi-commitment. The following definition fits to VSC based on either a random oracle or an ideal cipher.

Definition 1 (Vector Semi-commitment). *An IV-based vector commitment scheme VSC with message space $\mathcal{M}$ is defined by the following PPT algorithms.*

- **Commit**(iv, root) $\rightarrow$ (com, decom, $(m_0, \dots, m_{2^k-1})$): given an IV iv and a λ -bit string root, output a commitment com with opening information decom for messages $m = (m_0, \dots, m_{2^k-1}) \in \mathcal{M}^{2^k}$.
- **Open**(iv, decom, I) $\rightarrow$ pdecom: given an IV iv, opening information decom and a subset $I \subseteq [2^k]$ of indices, output a partial opening information pdecom for I.
- **Recon**(iv, pdecom, I) $\rightarrow ((m_i)_{i \in I}, \text{com})$: given an IV iv, a partial opening information pdecom for a subset I, output partially reconstructed messages and the full commitment com.
- **Verify**(iv, com, pdecom, I) $\rightarrow \{(m_i)_{i \in I}\} \cup \{\perp\}$: given an IV iv, a commitment com, a partial opening information pdecom, and a subset I, either output the messages $(m_i)_{i \in I}$ (accept) or $\perp$ (reject).

Although the interface is written for a general subset I, we will primarily consider I as a subset missing a single element unless otherwise specified. This opening is referred to as an *all-but-one*, with I being the all-but-one subset.

The first part of messages in vector semi-commitments are seeds in the signature scheme. As in the MPCitH-based signature schemes, if a prover can fix the sum of these seeds in a VOLEitH-based signature scheme, the prover can generate VOLE shares without $\mathbf{u}$ -correction. Specifically, if root is fixed, the first λ bits of the correction $\mathbf{c}_i$ of $\mathbf{u}$ is always zero.

Definition 2 (Correlated Vector Semi-commitment). *A vector semi-commitment scheme (Commit, Open, Recon, Verify) is correlated if*

$$\text{Commit}(\text{iv}, \text{root}) = (\text{com}, \text{decom}, (m_0, \dots, m_{2^k-1}))$$

implies $\text{msb}_\lambda(m_0 \oplus \dots \oplus m_{2^k-1}) = \text{root}$.

The extractable semi-binding property is a relaxation of extractable binding property. This property implies that while an adversary may find a small number of messages corresponding to the same commitment, it is hard to find a large number of such messages.

Definition 3 (Extractable Semi-binding). *Let VSC be an IV-based correlated vector semi-commitment scheme in the ideal cipher model with ideal cipher $E : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$. Let $\text{Ext}(\text{iv}, \mathcal{Q}, \text{com}) \rightarrow \{(m_i^{(j)})_{i \in [2^k]}\}_{j \in J}$ be a PPT algorithm that, given a set of query-response pairs of ideal cipher queries $\mathcal{Q}$, a commitment com , and outputs the committed messages $(m_i)_{i \in [2^k]}$. The u -extractable semi-binding game for VSC, denoted as $u\text{-ESB}$, with $2^k = \text{poly}(\lambda)$ and P queries to the ideal cipher and stateful adversary $\mathcal{A}$, is defined as follows.*

1. $(\text{iv}, \text{com}, \text{pdecom}_I, (m_i)_{i \in I}, I) \leftarrow \mathcal{A}^E(1^\lambda, P)$
2. $\{(m_i^{(j)})_{i \in [2^k]}\}_{j \in J} \leftarrow \text{Ext}(\text{iv}, \mathcal{Q}, \text{com})$, where $\mathcal{Q}$ is the set $\{(K_i, X_i, E_{K_i}(X_i))\}$ of query-response pairs of queries $\mathcal{A}$ made to E and $|J| \leq u$.
3. Output 1 if $\text{Verify}(\text{iv}, \text{com}, \text{pdecom}_I, I) \rightarrow (m_i^*)_{i \in I}$ and for all $j \in J$, there exists $i \in I$ such that $m_i^* \neq m_i^{(j)}$; otherwise, output 0.

We define $\mathcal{A}$'s u -extractable semi-binding advantage by

$$\text{Adv}_{\text{VSC}}^{u\text{-esb}}(\mathcal{A}) = \Pr[\mathcal{A} \text{ wins } u\text{-ESB}].$$

Although the extractor Ext extracts the set of messages $(m_i^{(j)})_{i \in [2^k], j \in J}$ in the definition, the purpose of the extractable semi-binding game is to bound the number of valid pdecoms for the same com . Therefore, the extractor will be programmed to extract the messages generated from valid pdecoms (see Supplementary Material B.1). We note that Ext may output $m_i = \perp$ if the committed value at index i is invalid.

The next property of vector semi-commitment is multi-instance hiding. The intuition behind the multi-instance hiding property is that given multiple puncturable PRF instances and their commitments, the punctured messages are indistinguishable from random.

Definition 4 (Multi-Instance Hiding). Let VSC be an IV-based correlated vector semi-commitment scheme in the ideal cipher model with ideal cipher E . The multi-instance hiding game for vector semi-commitments, Q_I -MIH, with $2^k = \text{poly}(\lambda)$ and P queries to the ideal cipher and stateful $\mathcal{A}$ is defined as follows.

1. $\text{root}_0, \text{root}_1 \leftarrow_{\$} \{0, 1\}^\lambda$, $b^* \leftarrow_{\$} \{0, 1\}$
2. For $j \in [Q_I]$, do the following:
 - (a) $\text{iv}_j \leftarrow_{\$} \{0, 1\}^{2\lambda}$
 - (b) $(\text{com}_j, \text{decom}_j, (m_{j,0}, \dots, m_{j,2^k-1})) \leftarrow \text{Commit}(\text{iv}_j, \text{root}_{b^*})$
 - (c) $\bar{i}_j \leftarrow_{\$} [2^k]$, $I_j = [2^k] \setminus \{\bar{i}_j\}$
 - (d) $\text{pdecom}_j \leftarrow \text{Open}(\text{iv}_j, \text{decom}_j, I_j)$
 - (e) If $b^* = 1$, set $m_{j,\bar{i}_j} \leftarrow_{\$} \mathcal{M}$
 - (f) If $b^* = 1$, set $m_{j,\bar{i}_j}[:\lambda] \leftarrow \text{root}_0 \oplus \bigoplus_{i \in [2^k] \setminus \{\bar{i}_j\}} m_{j,i}[:\lambda]$
3. $b \leftarrow \mathcal{A}^E(\text{iv}_j, (m_{j,i}[:\lambda])_{i \in [2^k]}, \text{pdecom}_j, I_j)_{j \in [Q_I]}$.
4. Output 1 if $b = b^*$, else 0.

We define $\mathcal{A}$'s multi-instance hiding advantage by

$$\text{Adv}_{\text{VSC}}^{Q_I\text{-mih}}(\mathcal{A}) = \left| \Pr[\mathcal{A} \text{ wins } Q_I\text{-MIH}] - \frac{1}{2} \right|.$$

We call VSC is multi-instance hiding if advantage of any Q_I -MIH adversary for polynomially bounded Q_I is negligible.

We note that Step 2f does not affect the whole game since $m_{j,\bar{i}_j}[:\lambda]$ is not given to the adversary. If Step 2f is absent, then the whole signature scheme cannot be correctly verified since root_0 works as (a part of) the real secret key in the EUF-CMA game.

3.1 Instantiation from Ideal Cipher

Here we briefly introduce IC-VSC. The following is essentially the same as IC-VSC in [19], but we slightly modify it to fit into the VOLE-in-the-Head framework. Let $E : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be a ideal cipher, and $\sigma : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be an orthomorphism. Then IC-VSC is constructed as Figure 1.

In Figure 1, there are descriptions of 4 algorithms: Commit^E , Open^E , Recon^E , and Verify^E . The Commit^E function generates a GGM tree from root , commit to the leaf nodes, and generate shares of $\mathbf{u}$ – we will call it *VOLE shares* – from the leaf nodes. The Open^E function opens the copath and the leaf commitment corresponding to a hidden leaf nodes. The Recon^E function reconstructs the GGM tree without the path to the hidden node and all the leaf commitments. The Verify^E function reconstructs as the Recon^E function, and verifies the reconstructed leaf commitments are valid with respect to the given commitment. Basically these functions have the same functionalities with corresponding MPCitH versions in [19], but Commit^E has little differences:

- **Parameters:** an ideal cipher E , an orthomorphism σ , integers k , bit-length of VOLE $\hat{\ell}$, and block length $\hat{\ell}_{\text{blk}} = \lceil \hat{\ell}/\lambda \rceil$.
- **Inputs:** $\text{iv} = (\text{iv}_1, \text{iv}_2, \mathbf{b}) \in \{0, 1\}^{\lambda+\lambda+8}$, and $\text{root} \in \{0, 1\}^\lambda$.
- **Commit^E(iv, root):**
 1. Set $\text{node}_{0,0} \leftarrow \text{root}$.
 2. For each level $e \in [k]$ and $i \in [2^e]$, set

$$\text{node}_{e+1,2i} \leftarrow \sigma(\text{node}_{e,i}) \oplus E_{\text{iv}_2}(\text{node}_{e,i} \oplus \text{iv}_1)$$

$$\text{node}_{e+1,2i+1} \leftarrow \text{node}_{e+1,2i} \oplus \text{node}_{e,i}.$$
 3. For $i \in [2^k]$, set

$$\text{seed}_i \leftarrow \text{node}_{k,i}$$

$$\text{ctr}[\mathbf{b}, i, 0] \leftarrow \text{iv}_1[: \lambda - 32] \parallel \mathbf{b} \parallel \langle i \rangle_{2B} \parallel \langle 0 \rangle_B$$

$$\text{com}_i \leftarrow E_{\text{seed}_i}(\text{ctr}[\mathbf{b}, i, 0])$$
 - (a) For $j \in [\hat{\ell}_{\text{blk}}]$,

$$\text{ctr}[\mathbf{b}, i, j+1] \leftarrow \text{iv}_1[: \lambda - 32] \parallel \mathbf{b} \parallel \langle i \rangle_{2B} \parallel \langle j+1 \rangle_B$$

$$\text{share}_{i,j} \leftarrow E_{\text{seed}_i}(\text{ctr}[\mathbf{b}, i, j+1])$$

If $j = \hat{\ell}_{\text{blk}} - 1$, then $\text{share}_{i,j} \leftarrow E_{\text{seed}_i}(\text{ctr}[\mathbf{b}, i, j+1])[: \hat{\ell} - (\hat{\ell}_{\text{blk}} - 1)\lambda]$
 - (b) Set $\text{share}_i \leftarrow \text{share}_{i,0} \parallel \dots \parallel \text{share}_{i,\hat{\ell}_{\text{blk}}-1}$ and $m_i \leftarrow \text{seed}_i \parallel \text{share}_i$.
- 4. Output a commitment $\text{com} = (\text{com}_0, \dots, \text{com}_{2^k-1})$ with opening information $\text{decom} = ((\text{node}_{e,i})_{e \in [k-1], i \in [2^{k-1}]}, \text{com})$, and messages $(m_0, \dots, m_{2^k-1})$.
- **Open^E(iv, decom, $I = [2^k] \setminus \{\bar{i}\}$):**
 1. Set $\text{path}_I \leftarrow (\text{node}_{k-e,i_e})_{e \in [k]}$ where $i_e = \lceil \bar{i}/2^e \rceil \oplus 1$ for $e \in [k]$;
 2. Output $\text{pdecom} = (\text{path}_I, \text{com}_{\bar{i}})$
- **Recon^E(iv, pdecom, $I = [2^k] \setminus \{\bar{i}\}$):**
 1. Similarly as Step 2 in Commit, expand each node in path_I and get $(\text{node}_{k,i})_{i \in I}$.
 2. For $i \in I$, do Step 3 in Commit^E.
 3. Output $((m_i)_{i \in I}, \text{com})$.
- **Verify^E(iv, $\text{com} = (\text{com}_i^*)_{i \in [2^k]}$, pdecom, $I = [2^k] \setminus \{\bar{i}\}$):**
 1. Similarly as Step 2 in Commit, expand each node in path_I and get $(\text{node}_{k,i})_{i \in I}$.
 2. For $i \in I$, do Step 3 in Commit^E.
 3. Output $(m_i)_{i \in I}$ if $\text{com} = \text{com}_i^*$ for all $i \in I$, or output $\perp$ otherwise.

Fig. 1: IC-VSC

- Since the number of leaf nodes 2^k may be larger than 2^8 in VOLEitH-based signature schemes, we allocate 2 bytes for the counter i in $\text{ctr}[\mathbf{b}, i, j + 1]$;
- As the length of VOLE shares is not necessarily a multiple of λ , we truncate the final block of VOLE share $\text{share}_{i, \ell_{\text{blk}} - 1}$.

The IC-VSC scheme will be used to simulate SoftSpoken VOLEs in the next section. In the following, we state the result of the extractable semi-binding property and the multi-instance hiding property of IC-VSC. Compared to the constructions of IC-VSC in [19], as the vector size 2^k becomes larger so that $\mathbf{b} \parallel \langle i \rangle_{2B} \parallel \langle \cdot \rangle_B$ in ctr occupy 32 bits instead of 24 bits. However, this does not affect the bound, since the number of multi-collisions on $\text{iv}_1[: \lambda - 32]$ will remain sufficiently small.

Lemma 1 (Extractable Semi-binding, [19]) *Let $E : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be an ideal cipher. Let $\mathcal{A}$ be an arbitrary adversary that makes P queries to E . Then, $\mathcal{A}$'s u -extractable semi-binding advantage $\text{Adv}_{\text{IC-VSC}}^{u\text{-esb}}(\mathcal{A})$ against IC-VSC is bounded by*

$$\text{Adv}_{\text{IC-VSC}}^{u\text{-esb}}(\mathcal{A}) \leq \frac{14P}{2^\lambda},$$

$$\text{for } u = 2^{k+1} \left(\frac{\lambda}{\log \lambda} \right)^2.$$

The proof of Lemma 1 is basically same as in that of Lemma 6 in [19], but we provide the proof in Supplementary Material B.1 for completeness. In the proof, there are two bad events as follows: informally,

- **BadE₁**: there exists two distinct height-1 nodes n and n' in the same position with identical leaf commitments;
- **BadE₂**: there exists a multi-collision of leaf commitments where the number of multi-collision is larger than $\nu = 2\lambda / \log \lambda$.

We note that a u -esb adversary can win only if one of these bad events happens. That is, there exists an extractor such that

$$\begin{aligned} \Pr [\text{BadE}_1 \vee \text{BadE}_2] &\leq \frac{14P}{2^\lambda}, \\ \Pr [\mathcal{A} \text{ wins } u\text{-esb} \wedge \neg(\text{BadE}_1 \vee \text{BadE}_2)] &= 0. \end{aligned} \tag{3}$$

This fact will be used in the proof of Lemma 3.

Let $\mathcal{L}_E$ be the set of query-response pair made by a u -esb adversary to E . For arbitrary fixed extraction $\mathcal{M} = \text{Ext}(\text{iv}, \mathcal{L}_E, \text{com})$, we have following property when $\neg(\text{BadE}_1 \vee \text{BadE}_2)$ holds.

Property 1. For distinct $(m_i)_{i \in [2^k]}, (m'_i)_{i \in [2^k]} \in \mathcal{M}$, let

$$\begin{aligned} \mathcal{I} &= \{ \Delta \in [2^k] : \exists \text{pdecom}, \text{IC-VSC.Verify}(\text{iv}, \text{com}, \text{pdecom}, [2^k] \setminus \{\Delta\}) = (m_i)_{i \neq \Delta} \}, \\ \mathcal{I}' &= \{ \Delta' \in [2^k] : \exists \text{pdecom}', \text{IC-VSC.Verify}(\text{iv}, \text{com}, \text{pdecom}', [2^k] \setminus \{\Delta'\}) = (m'_i)_{i \neq \Delta'} \}, \\ \mathcal{I}'' &= \mathcal{I} \cup \mathcal{I}', \end{aligned}$$

where pdecom and pdecom' should be able to computed from $\mathcal{L}_E$. Then, for $i \in [2^k]$ such that $i \notin \mathcal{I}''$ and $i \oplus 1 \notin \mathcal{I}''$, we have $m_i = m'_i$.

Property 1 means that two distinct extraction should be same except punctured leaf nodes and their siblings. It is because nodes with height ≥ 1 have corresponding leaf commitments of total length $\geq 2\lambda$. Therefore, even for VSC schemes, it becomes hard to find a collision between nodes with height ≥ 1 . This property will be used in the proof of Lemma 3.

Now we state the multi-instance hiding property of IC-VSC. We note that $\hat{\ell}_{\text{blk}} + 1$ is the block length of VOLE shares (including the seed block). Therefore, in the signature scheme, it will be set as $\hat{\ell}_{\text{blk}} = \lceil (\ell + \lambda + B)/\lambda \rceil$ where ℓ is the bit-length of the witness and B is a parameter related to VOLEHash . Similarly as extractable semi-binding property, the proof is a direct analogue of Lemma 7 in [19]. But we provide the proof in Supplementary Material B.1 for completeness.

Lemma 2 (Multi-instance Hiding, [19]) *Let $E : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be an ideal cipher. Let $\mathcal{A}$ be an arbitrary adversary that makes P queries to E . Let 2^k as the vector size. Provided that $Q_I \leq 2^{80}$, $\hat{\ell}_{\text{blk}} \leq 2^{16}$, and $k \leq 16$, $\mathcal{A}$'s multi-instance hiding advantage $\text{Adv}_{\text{IC-VSC}}^{Q_I\text{-mih}}(\mathcal{A})$ against IC-VSC is bounded by*

$$\text{Adv}_{\text{IC-VSC}}^{Q_I\text{-mih}}(\mathcal{A}) \leq \frac{(3 + 5 \cdot 2^{2k} + (\hat{\ell}_{\text{blk}} + 7)^2 + k)Q_I}{2^\lambda} + \frac{(25 + 24k)P}{2^\lambda} + \frac{2}{2^\lambda}.$$

4 Application of VSC to VOLEitH

In this section, we introduce a VSC-applied version of VOLE-in-the-Head paradigm, dubbed *reduced VOLEitH* ($r\text{VOLEitH}$). Essentially, the difference between VOLEitH and $r\text{VOLEitH}$ is the replacement of VC by VSC. We place changed algorithms from VOLEitH in this section, while unchanged algorithms are written in Supplementary Material A for completeness. In the description of $r\text{VOLEitH}$, we mostly follow the notations of FAEST [4] for readability, while some notations are changed.

4.1 Specification of $r\text{VOLEitH}$

In this section, we explain which parts of $r\text{VOLEitH}$ are changed from VOLEitH. As **Sign** and **Verify** algorithms are not so much different from those of VOLEitH, we provide the whole algorithm in Supplementary Material A.1. To summarize, there are three majorly changed parts: **VOLECommit**/**VOLEReconstruct**, functions related to the one-way function (OWF), and the use of grinding technique. We will explain the change individually.

The **VOLECommit** and **VOLEReconstruct** algorithms are provided in Algorithm 1 and Algorithm 2 respectively. The main differences is the replacement of VC by IC-VSC. Application of IC-VSC reduces the size of the leaf commitment $\overline{\text{com}}_{i,j}$ from 2λ bits to λ bits. Since IC-VSC uses correlated GGM tree,

now **VOLECommit** takes the extended witness $\mathbf{w}$ as input. The foremost λ bits are used as the root of the trees. For a minor difference, pseudorandom generation of VOLE shares was in **ConvertToVOLE** originally, but it is now in the **IC-VSC.Commit**. So, **ConvertToVOLE** (see Algorithm 3) now takes VOLE shares as inputs, not seeds, and it does not use PRG. Although we only use IC-VSC in this paper, we believe that security of an RO-VSC [19] version can be proved in a more easy way.

Algorithm 1. **VOLECommit**($\mathbf{w}[: \lambda], \text{iv}, \hat{\ell}; \text{param}$)

```

1  $(\text{iv}_0, \dots, \text{iv}_{\tau-1}) \in (\{0, 1\}^{2\lambda})^\tau \leftarrow H_4(\text{iv})$ 
2 for  $i \in [\tau]$  do
3   if  $i < \tau_0$  then  $b \leftarrow 0$  else  $b \leftarrow 1$ 
4    $(\text{com}_i, \text{decom}_i, (m_{i,0}, \dots, m_{i,2^{k_b}-1})) \leftarrow \text{IC-VSC.Commit}(\text{iv}_i, \mathbf{w}[: \lambda])$ 
5    $(\mathbf{u}_i, \mathbf{v}_{i,0}, \dots, \mathbf{v}_{i,k_b-1}) \leftarrow \text{ConvertToVOLE}(m_{i,0}, \dots, m_{i,2^{k_b}-1}, \text{iv}_i; \hat{\ell})$ 
6    $\mathbf{V}_i \leftarrow [\mathbf{v}_{i,0} \dots \mathbf{v}_{i,k_b-1}]$ 
7  $\mathbf{V} \leftarrow [\mathbf{V}_0 \dots \mathbf{V}_{\tau-1}] \in \mathbb{F}_2^{\hat{\ell} \times \lambda}$ 
8  $\mathbf{u} \leftarrow \mathbf{u}_0$ 
9 for  $i \in [\tau]^*$  do
10   $\mathbf{c}_i \leftarrow \mathbf{u}[\lambda :] \oplus \mathbf{u}_i[\lambda :]$ 
11  $h_{\text{com}} \leftarrow H_1(\text{com}_0 \| \dots \| \text{com}_{\tau-1})$ 
12 return  $(h_{\text{com}}, \text{decom}_0, \dots, \text{decom}_{\tau-1}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \mathbf{u}, \mathbf{V})$ 

```

We generalize functions for the proof of OWF from the proof of AES. The purpose of this change is to show that our VSC works regardless of the choice of OWF. These functions are not essential in this paper, and we just adopt the original work in a black-box manner. To be clear, we abstractly describe functions **OWFProve** and **OWFVerify** in Supplementary Material A.

The FAESTER signature scheme [2] – an improvement of FAEST – uses the grinding technique, which requires the last w bits of the output of H_2^3 should all zeros. This can reduce the parameter τ_i or k_i so that the corresponding signature scheme become more efficient. We incorporate the grinding technique in **rVOLEitH**.

There is also a minor change. As IC-VSC takes 2λ bits salt as input, we generates 2λ -bit iv rather than 128 bits, and stretch the random strings to $2\tau\lambda$ bits in **VOLECommit** and **VOLEReconstruct**. When stretching iv , we use a new hash function $H_4 : \{0, 1\}^{2\lambda} \rightarrow (\{0, 1\}^{2\lambda})^\tau$.

We denote **param** for the parameter set of **rVOLEitH** itself, which contains $(\lambda, \ell, \tau, \tau_0, \tau_1, k_0, k_1, B, w)$ where ℓ is the length of extended witness. The definitions of each parameters can be found in the next subsection. On the other hand, we also denote **param_{OWF}** for the parameter set of OWF, but what it contains will be different for each OWFs.

Algorithm 2. VOLEReconstruct(chall, (pdecom_i)_{i∈[τ]}, iv, $\hat{\ell}$; param)

```

1 (iv0, ..., ivτ-1) ∈ ({0, 1}2λ)τ ← H4(iv)
2 for i ∈ [τ] do
3   if i < τ0 then b ← 0 else b ← 1
4   (δi,0, ..., δi,kb-1) ← ChalDec(chall, i; param)
5   Δi ← NumRec(kb, (δi,0, ..., δi,kb-1))
6   (comi, (m'i,j)j∈[2kb]\{i}) ← IC-VSC.Recon(pdecomi, [2kb]\{Δi, (ivi, i))
7   for j ∈ [2kb]* do
8     mi,j ← m'i,j ⊕ Δi
9   (u'i, qi,0, ..., qi,kb-1) ← ConvertToVOLE(⊥, mi,0, ..., mi,2kb-1;  $\hat{\ell}$ )
10  Q'i ← [qi,0 ... qi,kb-1]
11 hcom ← H1(com0 || ... || comτ-1)
12 return (hcom, Q'0, ..., Q'τ-1)

```

Algorithm 3. ConvertToVOLE(m₀, ..., m_{2^k-1}; $\hat{\ell}$)

```

1 if m0 = ⊥ then r0,0 ← 0 $\hat{\ell}$  else r0,0 ← m0
2 for i ∈ [2kb]* do
3   r0,i ← mi
4 v0, ..., vk-1 ← 0 $\hat{\ell}$ 
5 for j ∈ [k] do
6   for i ∈ [2k-j-1] do
7     vj ← vj ⊕ rj,2i+1
8     rj+1,i ← rj,2i ⊕ rj,2i+1
9 u ← rk,0
10 return (u, v0, ..., vk-1)

```

4.2 Application to Signature Schemes

In this section, we apply the rVOLEitH paradigm to concrete VOLEitH-based signature schemes. Optimization techniques such as cGGM tree [15] and grinding technique [2] are applied to our reduced versions of signature schemes, similarly to the signature scheme in [19].

Performance Comparison. In Table 1, we compare the signature size and computational cost between original VOLEitH-based signature schemes and reduced versions of them. Similarly as rVOLEitH, we append “r” to the name of reduced version of signature schemes. We choose AES versions of FAESTER [2], not an Even-Mansour version. For KuMQuat and MandaRain [2], we use parameter sets KuMQuat-2¹ and MandaRain-4, respectively. DSDitH refers to dual sup-

Trade-Off	Scheme	Scheme Parameters					Computational Cost			Sig. (B)	Red. Rate
		τ	$(\tau_0, 2^{k_0})$	$(\tau_1, 2^{k_1})$	B	T_{open}	#Leaves	$\log \theta$	w		
Short	FAESTER	11	$(0, 2^{12})$	$(11, 2^{11})$	16	102	22528	5.4	7	4594	-
	KuMQuat	11	$(7, 2^{12})$	$(4, 2^{11})$	16	99	36864	7.2	7	2555	-
	MandaRain	11	$(7, 2^{12})$	$(4, 2^{11})$	16	101	36864	6.0	7	3082	-
	DSDitH	11	$(0, 2^{12})$	$(11, 2^{11})$	16	99	22528	7.2	7	2851	-
	rFAESTER	11	$(6, 2^{11})$	$(5, 2^{10})$	48	116	17408	0	12	4526	1.5%
	rKuMQuat	11	$(0, 2^{12})$	$(11, 2^{11})$	48	121	22528	0	14	2503	2.0%
	rMandaRain	11	$(1, 2^{12})$	$(10, 2^{11})$	48	122	24576	0	13	3126	-1.4%
	rDSDitH	11	$(4, 2^{11})$	$(7, 2^{10})$	48	114	15360	0	14	2783	2.4%
Fast	FAESTER	16	$(8, 2^8)$	$(8, 2^7)$	16	110	3072	0.5	8	6052	-
	KuMQuat	16	$(0, 2^8)$	$(16, 2^8)$	16	102	4096	4.5	4	3028	-
	MandaRain	16	$(0, 2^8)$	$(16, 2^8)$	16	110	4096	0.6	8	3876	-
	DSDitH	16	$(8, 2^8)$	$(8, 2^7)$	16	102	3072	2.9	8	3450	-
	rFAESTER	16	$(7, 2^8)$	$(9, 2^7)$	40	119	2944	0	9	5748	5.0%
	rKuMQuat	16	$(11, 2^8)$	$(5, 2^7)$	40	119	3456	0	9	2852	5.8%
	rMandaRain	16	$(15, 2^8)$	$(1, 2^7)$	40	127	3968	0	9	3710	4.2%
	rDSDitH	16	$(6, 2^8)$	$(10, 2^7)$	40	118	2816	0	11	3242	6.0%

Table 1: Performance comparison between (r)VOLEitH-based signature schemes. If reduction rate is positive, then rVOLEitH-based signature scheme is smaller than the corresponding counterpart.

port decomposition in the head proposed by Bidoux et al. [6], where the data in the table is the VOLEitH version, not the TCitH version.

For parameters, τ is the number of repetitions of GGM tree evaluation. 2^{k_0} and 2^{k_1} are the numbers of GGM tree leaf nodes. τ_i is the number of repetitions with GGM tree of 2^{k_i} leaf nodes, where $\tau = \tau_0 + \tau_1$. Parameter B is the bit-length of additional VOLEHash input to achieve hiding property. The reduced versions require little bit larger B since there are more than one preimage copath to be VOLEHashed. We refer to the next section for more details. T_{open} is the number of parameter nodes to be opened. Since the reduced versions are not involved with the one-tree technique at this moment, T_{open} of them can be computed as $\tau_0 k_0 + \tau_1 k_1$.

To compare computational cost of one-tree VOLEitH and “several-tree” rV-OLEitH, we provide three indicators. “#Leaves” is the total number of leaf nodes in the GGM tree(s). Signing and verification time is roughly proportional to this indicator. θ was proposed in [6] to measure the effect of one-tree technique to signing time. Let p to be the probability that the number of opened nodes is smaller than or equal to T_{open} for a single H_2^3 query. Then θ is defined by $\theta = 1/p$, which means the average number to pass the third challenge chall_3 . w is the number of bits to be zero in the grinding technique. Putting it all together, a single signature requires expectedly $\theta \cdot 2^w$ evaluations of H_2^3 . To make a fair

comparison, we set w of the reduced versions to be roughly same as $\log \theta + w$ of original VOLEitH-based signature schemes.

Signature size of original VOLEitH-based signature scheme is (in bits)

$$\begin{aligned} \text{Size} &= \underbrace{(\tau - 1)(\ell + 2\lambda + B)}_{(\mathbf{c}_i)_i} + \underbrace{\lambda + B + \ell + \lambda}_{\widetilde{\mathbf{u}}, \widetilde{\mathbf{d}}, \widetilde{\mathbf{a}}} + \underbrace{\tau_0(k_0\lambda + 2\lambda) + \tau_1(k_1\lambda + 2\lambda)}_{(\mathbf{pdecom}_i)_i} + \underbrace{\lambda + \ell_{iv}}_{\mathbf{chall}_3, iv} \\ &= \tau \cdot (\ell + 4\lambda + B) + (\tau_0 k_0 + \tau_1 k_1)\lambda + \lambda + \ell_{iv}. \end{aligned}$$

If the one-tree technique is applied, then the signature size becomes

$$\tau \cdot (\ell + 4\lambda + B) + T_{\text{open}}\lambda + \lambda + \ell_{iv} + 4.$$

For the rVOLEitH-based signature schemes, the signature size is

$$\begin{aligned} \text{Size} &= (\tau - 1)(\ell + \lambda + B) + \lambda + B + \ell + \tau_0(k_0\lambda + \lambda) + \tau_1(k_1\lambda + \lambda) + \lambda + \ell_{iv} + 4 \\ &= \tau \cdot (\ell + \lambda + B) + (\tau_0 k_0 + \tau_1 k_1)\lambda + \lambda + \ell_{iv} + 4. \end{aligned}$$

If grinding or one-tree technique is used, we added 4 bytes for the counter.

In Table 1, the rVOLEitH-based signature schemes show higher reduction rate from 4.2% to 6.0% in the fast parameter sets, compared to even from -1.4% to 2.4% in the short parameter sets. This is because the signature size reducing effect of root-fixed cGGM is better at larger τ , while that of the one-tree technique seems the opposite. From this table, we may expect that the combination of one-tree and VSC is better than the current rVOLEitH in the short parameter sets.

To compare the real speed of signing/verification algorithms, we provide the timing results of signing/verification algorithms of FAESTER and rFAESTER in Table 2. We estimated timing result of an AVX2 version of rFAESTER upon the the one-tree repository³ using “ccec” option. We note that it is slightly different from the specification in Section 4.1; it uses normal GGM rather than cGGM, and iv in the counter for AES input is XORed rather than concatenated. We believe that this difference is not advantageous for rFAESTER. The experiment is done in AMD Ryzen 7 2700X 3.7 GHz with 128 GB RAM.

In Table 2, rFAESTER enjoys $2.2\times$ - $3.0\times$ faster signing and $5.2\times$ - $7.4\times$ faster verification, which are much better results than theoretically estimated in Table 1. This is because FAESTER uses a heavy hash function to compute leaf commitments, while rFAESTER uses AES to compute leaf commitments.

Application to One-Tree Technique. Our security proof of rVOLEitH heavily depends on the shape of $\mathbf{pdecom}_i$, which includes a commitment and a copath of a hidden node in a GGM tree. We utilize the fact that a copath contains only one leaf node so that any set of commitments $(\widetilde{\mathbf{com}}_{i,j})_j$ does not have too much preimage copath. As nodes with height ≥ 1 can collide in commitments with negligible probability, the number of preimage copath depends on the number of leaf nodes contained in $\mathbf{pdecom}_i$.

³ <https://github.com/faest-sign/faest-one-tree>

	Scheme	Sign (ms)	Verify (ms)
Short	FAESTER	11.27	19.15
	rFAESTER	$\simeq 3.66$	$\simeq 2.56$
Fast	FAESTER	1.46	2.62
	rFAESTER	$\simeq 0.65$	$\simeq 0.50$

Table 2: Speed comparison between FAESTER and rFAESTER. The $\simeq$ means estimated by a similar algorithm. The parameters are given in Table 1.

However, one-tree technique proposed by Baum et al. [2] join all the τ trees to one tree so that the shape of copath – it may be inaccurate to call this a copath, but we will call it that here – is not fixed; a copath may have too many leaf nodes. At this moment, we do not know whether one-tree technique can be securely combined with a vector semi-commitment, and we leave it as a future work.

5 Security Proof of rVOLEitH

For simplicity, we assume that $N = 2^{k_0} = 2^{k_1}$ and ℓ is a multiple of λ , but we believe this can be easily adapted to general parameters. We also omit some notations such as `param`, `paramOWF` if the context is clear.

5.1 Security of VOLECommit + VOLEConsistencyCheck.

In this section, we define useful helper functions, dubbed `ChosenVOLECommit` and `ChosenVOLEReconstruct`, which can be understood as composition of `VOLECommit` and `VOLEReconstruct`, respectively, with a VOLE consistency check. Algorithmic descriptions of `ChosenVOLECommit` and `ChosenVOLEReconstruct` are provided in Algorithm 4 and Algorithm 5.

In Algorithm 4, `ChosenVOLECommit` commits to a random VOLE correlation $(\mathbf{u}, \mathbf{V})$, checks its consistency using `VOLEHash`, and commits to the witness. The main point of this function is that an adversary is hard to find a VOLE correlation which results in matching `chall1` and h_V unless the adversary generated `chall1` and h_V legitimately from $(\mathbf{u}, \mathbf{V})$. In Algorithm 5, `ChosenVOLEReconstruct` reconstructs a random VOLE correlation from a given partial decommitment, checks its consistency, and outputs the corresponding `chall1` and h_V .

In the original VOLEitH paradigm, the extractable binding property of `VOLECommit` ensures EUF-KO security. However, in rVOLEitH utilizing VSC, `VOLECommit` lacks extractable binding and instead achieves extractable semi-binding. We demonstrate that `VOLECommit` regains full extractable binding when combined with the VOLE consistency check. More precisely, `ChosenVOLECommit` satisfies the binding properties required for the EUF-KO security proof.

Algorithm 4. ChosenVOLECommit($\mu, \mathbf{w}, \text{iv}$)

```

// Commitment and VOLE derandomization
1  $(h_{\text{com}}, (\text{decom}_i)_{i \in [\tau]}, (\mathbf{c}_i)_{i \in [\tau]^*}, \mathbf{u}, \mathbf{V}) \leftarrow \text{VOLECommit}(\mathbf{w}, \text{iv}, \ell + 2\lambda + B).$ 
2  $\text{chall}_1 \leftarrow H_2^1(\mu \| h_{\text{com}} \| \mathbf{c}_1 \| \dots \| \mathbf{c}_{\tau-1} \| \text{iv}; 5\lambda + 64)$ 

// VOLE consistency check and witness commitment
3  $\tilde{\mathbf{u}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{u})$ 
4  $\tilde{\mathbf{V}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{V})$ 
5  $h_V \leftarrow H_1(\tilde{\mathbf{V}})$ 
6  $\mathbf{d} \leftarrow \mathbf{w}[\lambda : ] \oplus \mathbf{u}[\lambda : \ell]$ 
7 return  $(\text{chall}_1, (\text{decom}_i)_{i \in [\tau]}, (\mathbf{c}_i)_{i \in [\tau]^*}, \mathbf{u}, \mathbf{V}, \tilde{\mathbf{u}}, h_V, \mathbf{d})$ 

```

From VSC Extractor to ChosenVOLECommit Extractor. To prove the extractable binding property of ChosenVOLECommit, an extractor is necessary. We will construct a ChosenVOLECommit extractor Ext on top of VSC extractor Ext' (in Definition 3). At a high level, the ChosenVOLECommit extractor will try to extract a VOLE pair which also passes the VOLE consistency check from the adversarial queries. If there is no such VOLE pair or there are multiple VOLE pairs passes the VOLE consistency check, then the extractor returns $\perp$. When the extractor returns $\perp$, we will divide $\perp$ by $\perp_{\text{abort}}$ and $\perp_{\text{non}}$ for readability; $\perp_{\text{abort}}$ means that it will be an abort condition in the extractable binding proof, and $\perp_{\text{non}}$ means otherwise. Algorithmically, both $\perp_{\text{abort}}$ and $\perp_{\text{non}}$ returns nothing.

Fix an arbitrary $\mathcal{A}$ and let $\mathcal{L}_E, \mathcal{L}_1, \mathcal{L}_2^1$ and $\mathcal{L}_4$ are query-response pairs made by $\mathcal{A}$ to E, H_1, H_2^1 and H_4 , respectively. Then, given an input $(\text{chall}_1, \tilde{\mathbf{u}}, h_V, \mathcal{L} = (\mathcal{L}_E, \mathcal{L}_1, \mathcal{L}_2^1, \mathcal{L}_4))$, ChosenVOLECommit extractor Ext does the following.

1. Find a preimage $\mu \| h_{\text{com}} \| \mathbf{c}_1 \| \dots \| \mathbf{c}_{\tau-1} \| \text{iv}$ of chall_1 under H_2^1 from $\mathcal{L}_2^1$. If there is no unique preimage, Ext immediately returns $\perp_{\text{non}}$, otherwise, let $\mathbf{c}_0 = 0$.
2. Find a preimage $\tilde{\mathbf{V}}$ of h_V under H_1 from $\mathcal{L}_1$. If there is no unique preimage, Ext immediately returns $\perp_{\text{non}}$.
3. Find a preimage $(\overline{\text{com}}_{i,j})_{i \in [\tau], j \in [N]}$ of h_{com} under H_1 from $\mathcal{L}_1$. If there is no unique preimage, Ext immediately returns $\perp_{\text{non}}$.
4. Find an image $(\text{iv}_0, \dots, \text{iv}_{\tau-1})$ of iv under H_4 from $\mathcal{L}_4$. If there is no image, Ext immediately returns $\perp_{\text{non}}$.
5. For each $i \in [\tau]$, extract $\mathcal{M}_i$ using VSC extractor Ext' where

$$\mathcal{M}_i = \text{Ext}'((\text{iv}_i, i), \mathcal{L}_E, \text{com}_i = (\overline{\text{com}}_{i,j})_{j \in [N]}).$$

If there exists $i \in [\tau]$ such that $|\mathcal{M}_i| = 0$, Ext immediately returns $\perp_{\text{non}}$. Now, we have $0 < |\mathcal{M}_i| \leq u$.

Algorithm 5. ChosenVOLEReconstruct(chall₃, (pdecom_i)_{i∈[τ]}, (c_i)_{i∈[τ]*}, $\tilde{\mathbf{u}}, \mu, \text{iv}$)

```

// Reconstruct VOLEs and check commitment
1 (hcom, Q') ← VOLEReconstruct(chall3, (pdecomi)i∈[τ], iv, ℓ + 2λ + B).
2 chall1 ← H21(μ || hcom || c1 || ... || cτ-1 || iv; 5λ + 64)

// Apply the VOLE correction values and check consistency
3 Q0 ← Q'_0
4 for i ∈ [τ] do
5   (δ0, ..., δk-1) ← ChalDec(chall3, i)
6    $\tilde{\mathbf{D}}_i \leftarrow [\delta_0 \cdot \tilde{\mathbf{u}} \dots \delta_{k-1} \cdot \tilde{\mathbf{u}}] \in \{0, 1\}^{(\lambda+B) \times k}$ 
7   if i ≥ 1 then
8     ci ← 0λ || ci
9     Qi ← Q0 ⊕ [δ0 · ci ... δk-1 · ci]
10 Denote Q = [Q0 ... Qτ-1], and  $\tilde{\mathbf{D}} = [\tilde{\mathbf{D}}_0 \dots \tilde{\mathbf{D}}_{\tau-1}]$ 
11  $\tilde{\mathbf{Q}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{Q}) \in \{0, 1\}^{(\lambda+B) \times \lambda}$ 
12 hV ← H1( $\tilde{\mathbf{Q}} \oplus \tilde{\mathbf{D}}$ )
13 return (chall1, hV, Q)

```

6. For each $i \in [\tau]$ and $(m_j)_{j \in [N]} \in \mathcal{M}_i$, let $\mathcal{S}_i[(m_j)_{j \in [N]}]$ be the collection of $(\Delta, (m'_j)_{j \in [N]})$ such that

$$\begin{cases} m'_j = m_j \neq \perp \text{ for } j \neq \Delta \text{ and } m'_\Delta = \perp, \\ (\mathbf{u}, \mathbf{V}) := \text{ConvertToVOLE}(m_0, \dots, m_{N-1}), \\ \text{VOLEHash}(\text{chall}_1, \Delta[j] \cdot (\mathbf{u} \oplus \mathbf{c}_i) \oplus \mathbf{V}[j]) = \Delta[j] \cdot \tilde{\mathbf{u}} \oplus \tilde{\mathbf{V}}[ik + j] \text{ for } j \in [k]. \end{cases}$$

Define

$$\begin{aligned} \mathcal{S}_i^{(1)} &:= \{ \mathcal{S}_i[(m_j)_{j \in [N]}] : |\mathcal{S}_i[(m_j)_{j \in [N]}]| = 1, (m_j)_{j \in [N]} \in \mathcal{M}_i \}, \\ \mathcal{S}_i^{(2)} &:= \{ (\perp, (m_j)_{j \in [N]}) : |\mathcal{S}_i[(m_j)_{j \in [N]}]| \geq 2, (m_j)_{j \in [N]} \in \mathcal{M}_i \}. \end{aligned}$$

$\mathcal{S}_i^{(1)}$ is the set of extracted messages where Δ of the corresponding VOLE pair is uniquely determined, while Δ for $\mathcal{S}_i^{(2)}$ is not uniquely determined since there is no $\perp$ in $(m_j)_{j \in [N]}$. Ext immediately returns $\perp_{\text{abort}}$ if $|\mathcal{S}_i^{(2)}| \geq 2$ or $|\mathcal{S}_i^{(2)}| = 0$ and $|\mathcal{S}_i^{(1)}| \neq 1$. Then, either $\mathcal{S}_i^{(2)}$ has unique element, or $\mathcal{S}_i^{(2)}$ is empty but $\mathcal{S}_i^{(1)}$ has unique element. Let $(\Delta_i^*, (m_{i,j}^*)_{j \in [N]})$ is that unique element, and also remember the corresponding VOLE pair $(\mathbf{u}_i^*, \mathbf{V}_i^*)$ where $(\mathbf{u}_i^* \oplus \mathbf{c}_i, \mathbf{V}_i^*) = \text{ConvertToVOLE}((m_{i,j}^*)_{j \in [N]})$. We note that $\mathbf{u}_i^*$ is the corrected value by $\mathbf{c}_i$.

7. If there exist distinct $i, j \in [\tau]$, such that $\Delta_i^* = \Delta_j^* = \perp$ and $\mathbf{u}_i^* \neq \mathbf{u}_j^*$, Ext immediately returns $\perp_{\text{abort}}$. Then, all $\mathbf{u}_i^*$'s with $\Delta_i^* = \perp$ are identical, and let $\mathbf{u}^*$ for that value (if there is no such $\mathbf{u}_i^*$, set $\mathbf{u}^*$ as arbitrary value). Also,

compute $\mathbf{V}^*$ as following.

$$\begin{cases} \mathbf{V}^*[ik+j] = \mathbf{V}_i^*[j] & \text{if } \Delta_i^* = \perp \\ \mathbf{V}^*[ik+j] = \mathbf{V}_i^*[j] \oplus \Delta_i^*[j] \cdot (\mathbf{u}_i^* \oplus \mathbf{u}^*) & \text{otherwise} \end{cases}$$

Finally, Ext returns $(\mathbf{u}^*, \mathbf{V}^*, (\mathbf{c}_i)_{i \in [\tau]^*}, \mu, \text{iv})$.

From Step 1 to 5, Ext extracts committed messages. In Step 6, it extracts (unique) base VOLEs that passes the VOLE consistency check. In Step 7, Ext computes final output from the extracted base VOLEs.

Now we provide the definition of extractable binding property of ChosenVOLECommit in the following.

Lemma 3 (Extractable Binding of ChosenVOLECommit) *Let H_1, H_2^1 , and H_4 be random oracles, and E be an ideal cipher. Let Ext be the ChosenVOLECommit extractor defined above. Let $\mathcal{A}$ be a stateful adversary against the following extractable binding game with the number of queries $Q_1, Q_{2,1}, Q_4$, and P to H_1, H_2^1, H_4 , and E , respectively.*

- a) $(\text{chall}_1, \tilde{\mathbf{u}}, h_V) \leftarrow \mathcal{A}^{E, H_1, H_2^1, H_4}(\text{commit})$.
- b) $(\mathbf{u}^*, \mathbf{V}^*, (\mathbf{c}_i)_{i \in [\tau]^*}, \mu, \text{iv}) = \text{Ext}(\text{chall}_1, \tilde{\mathbf{u}}, h_V, \mathcal{L} = (\mathcal{L}_E, \mathcal{L}_1, \mathcal{L}_2^1, \mathcal{L}_4))$ where $\mathcal{L}$ is defined above.
- c) $(\text{chall}, (\text{pdecom}_i)_{i \in [\tau]}) \leftarrow \mathcal{A}^{E, H_1, H_2^1, H_4}(\text{open})$
- d) $(\text{chall}'_1, h'_V, \mathbf{Q}') \leftarrow \text{ChosenVOLEReconstruct}(\text{chall}, (\text{pdecom}_i)_{i \in [\tau]}, (\mathbf{c}_i)_{i \in [\tau]^*}, \tilde{\mathbf{u}}, \mu, \text{iv})$
- e) *Output 0 (failure) if*
 - $(\text{chall}'_1, h'_V) \neq (\text{chall}_1, h_V)$, or
 - $\mathbf{Q}'[ik+j] = \mathbf{V}^*[ik+j] \oplus \Delta_i[j] \cdot \mathbf{u}^*$ for all $i \in [\tau], j \in [k]$ where $\Delta_i = \text{ChalDec}(\text{chall}, i)$.
- f) *Output 1 (success) otherwise.*

Then there exists Γ_{Bad} (which will be defined in the proof) where

$$\Pr[\mathcal{L} \in \Gamma_{\text{Bad}}] \leq \frac{Q^2}{2^{2\lambda}} + \frac{18P}{2^\lambda} + \frac{\tau u^2 \varepsilon_v (k + \tau + 1) Q_{2,1}}{2} + \frac{8u^2 (k + \tau) Q_{2,1} P}{2^{2\lambda+B}}$$

and

$$\Pr[\mathcal{A} \text{ wins} \mid \mathcal{L} \notin \Gamma_{\text{Bad}}] \leq \frac{8uP}{2^{2\lambda+B}}$$

where $u = 2N(\lambda/\log \lambda)^2$, and $Q = Q_1 + Q_{2,1} + Q_4$.

Proof. Suppose that $\mathcal{A}$ makes an ideal cipher query and get (seed, X, Y) , i.e., $E_{\text{seed}}(X) = Y$. In the following, we denote $\ell_{\text{blk}} := \lceil (\ell + \lambda + B)/\lambda \rceil$, and $\text{ctr}[i, j, k] := \langle i \rangle_B \parallel \langle j \rangle_{2B} \parallel \langle k \rangle_B$. We define $\text{PRG}_{i,j}(\text{seed}, \text{iv})$ by

$$\text{seed} \parallel E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, 1]) \parallel \cdots \parallel E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, \ell_{\text{blk}}])[: B].$$

We note that $E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, 0])$ represents the leaf commitment of seed (with iv), and the $\text{PRG}_{i,j}(\text{seed}, \text{iv})$ represents the corresponding VOLE shares.

At a high level, by giving additional information to $\mathcal{A}$, we want $\mathcal{A}$ to know the entire PRG output blocks instead of just a single block. If $X = \text{iv} \parallel \text{ctr}[i, j, a]$ for some $\text{iv} \in \{0, 1\}^{\lambda-32}$, $i \in [\tau]$, $j \in [N]$, $a \in [\ell_{\text{blk}} + 1]$ is queried, we give a set of queries

$$\{(\text{seed}, X', Y') : Y' = E_{\text{seed}}(X'), X' = \text{iv} \parallel \text{ctr}[i, j, a'], a' \in [\ell_{\text{blk}} + 1]\} \quad (4)$$

instead of giving (seed, X, Y) alone to $\mathcal{A}$. By this modification, $\mathcal{A}$ can get at most P whole PRG outputs, which is $(\ell_{\text{blk}} + 1)P$ evaluations of E .

Now, we argue via game hops. Let G_0 be the original game. Intuitively, if Ext cannot extract multiple VOLE pairs and $\mathcal{A}$ cannot find an additional unextracted VOLE pair afterwards, then the extractable binding property is satisfied. G_1 and G_2 proactively prevents some bad events for the ease of probability computation in the following games. G_3 and G_4 abort if Ext can extract multiple VOLE pairs which passes the VOLE consistency check (i.e., Ext returns $\perp_{\text{abort}}$). G_5 bounds the probability that $\mathcal{A}$ finds another VOLE pair which is not extracted.

G_1 : This game aborts if there exists a collision, or preimage-finding event on the random oracles after the extraction; Ext returned $\perp_{\text{non}}$ as a collision or a preimage was not found, but $\mathcal{A}$ found the a collision or a preimage afterwards. Then,

$$\Pr[G_1 \text{ aborts}] \leq Q^2/2^{2\lambda}$$

where Q is total number of random oracle queries from $\mathcal{A}$.

G_2 : This game aborts if one of the following bad events happen. These events are prevented to compute the probability of $\perp_{\text{abort}}$ in Step 6 easily.

- **BadE₃**: There exists $\text{iv} \in \{0, 1\}^{\lambda-32}$, $i \in [\tau]$, and $(\text{seed}_a, j_a)_{a \in [4]} \in \{0, 1\}^\lambda \times [N]$ where $(j_0, j_1) = (j_2, j_3)$ such that

$$\begin{cases} E_{\text{seed}_0}(\text{iv} \parallel \text{ctr}[i, j_0, 0]) = E_{\text{seed}_2}(\text{iv} \parallel \text{ctr}[i, j_2, 0]), \\ E_{\text{seed}_1}(\text{iv} \parallel \text{ctr}[i, j_1, 0]) = E_{\text{seed}_3}(\text{iv} \parallel \text{ctr}[i, j_3, 0]), \\ \bigoplus_{a \in [4]} \text{PRG}_{i, j_a}(\text{seed}_a, \text{iv}) = 0; \end{cases}$$

- **BadE₄**: There exists $\text{iv} \in \{0, 1\}^{\lambda-32}$, $i \in [\tau]$, and $(\text{seed}_a, j_a)_{a \in [2]} \in \{0, 1\}^\lambda \times [N]$ such that

$$\text{PRG}_{i, j_0}(\text{seed}_0, \text{iv}) = \text{PRG}_{i, j_1}(\text{seed}_1, \text{iv}).$$

- **BadE₁ $\vee$ BadE₂** (in Section 3.1) occurs.

Then, as there are at most P of known PRG outputs, we have

$$\begin{aligned} \Pr[G_2 \text{ aborts}] &\leq \frac{P^4}{(2^\lambda - (\ell_{\text{blk}} + 1)P)^{2+(\ell_{\text{blk}}-1)}} + \frac{P^2}{(2^\lambda - (\ell_{\text{blk}} + 1)P)^{\ell_{\text{blk}}-1}} + \frac{14P}{2^\lambda} \\ &\leq \left(1 + \frac{P^2}{2^{2\lambda-2}}\right) \frac{P^2}{(2^{\lambda-1})^{\ell_{\text{blk}}-1}} + \frac{14P}{2^\lambda} \leq \frac{18P}{2^\lambda} \end{aligned}$$

provided that $(\ell_{\text{blk}} + 1)P \leq 2^{\lambda-1}$ and $\ell_{\text{blk}} \geq 3$.

G_3 : In this game, we will show that Ext does not return $\perp_{\text{abort}}$ in Step 6 unless an abort condition happens.

For the following games, assume that $\mu \parallel h_{\text{com}} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}$ is queried to H_2^1 by $\mathcal{A}$ and extraction succeeds for all $i \in [\tau]$, i.e., $\mathcal{M}_i \neq \emptyset$ for all $i \in [\tau]$. By (3), we can sure that $\mathcal{M}_i$ consists of all possible messages at the time of extraction. Let $\mathcal{E}_i$ be the extracted base VOLEs, i.e.,

$$\mathcal{E}_i = \{(\mathbf{u}, \mathbf{V}) = \text{ConvertToVOLE}(m_0, \dots, m_{N-1}) : (m_0, \dots, m_{N-1}) \in \mathcal{M}_i\}.$$

The game aborts when two different base VOLEs in one tree both pass the consistency check, except one case explained below. If two distinct base VOLEs $(\mathbf{u}, \mathbf{V}), (\mathbf{u}', \mathbf{V}') \in \mathcal{E}_i$ pass the consistency check, there should be $\Delta = (\delta_0, \dots, \delta_{k-1}) \in \{0, 1\}^k$ and $\Delta' = (\delta'_0, \dots, \delta'_{k-1}) \in \{0, 1\}^k$ such that

$$\begin{cases} (\text{VOLEHash}(\text{chall}_1, \mathbf{u} \oplus \mathbf{c}_i) \oplus \tilde{\mathbf{u}}) \delta_j = \text{VOLEHash}(\text{chall}_1, \mathbf{V}[j]) \oplus \tilde{\mathbf{V}}[j] \\ (\text{VOLEHash}(\text{chall}_1, \mathbf{u}' \oplus \mathbf{c}_i) \oplus \tilde{\mathbf{u}}) \delta'_j = \text{VOLEHash}(\text{chall}_1, \mathbf{V}'[j]) \oplus \tilde{\mathbf{V}}[j] \end{cases} \quad (5)$$

for all $j \in [k]$. In this game, we will prevent the case that (5) has a solution by eliminating the event that at least one of following conditions hold.

1. If $\delta_0 = \delta'_0 = 0$, $\text{VOLEHash}(\text{chall}_1, \mathbf{V}[0] \oplus \mathbf{V}'[0]) = 0$.
2. If $\delta_0 = \delta'_0 = 1$, $\text{VOLEHash}(\text{chall}_1, \mathbf{u} \oplus \mathbf{V}[0] \oplus \mathbf{u}' \oplus \mathbf{V}'[0]) = 0$.
3. If $\delta_0 \oplus \delta'_0 = \delta_j \oplus \delta'_j = 1$ for some $j \in \{1, \dots, k-1\}$,

$$\text{VOLEHash}(\text{chall}_1, \mathbf{V}[0] \oplus \mathbf{V}[j] \oplus \mathbf{V}'[0] \oplus \mathbf{V}'[j]) = 0.$$

We note that the above events does not prevent one case $\Delta \oplus \Delta' = 1$, which will be addressed in a later game. To sum up, the game aborts if there exists $i \in [\tau]$ and distinct $(\mathbf{u}, \mathbf{V}), (\mathbf{u}', \mathbf{V}') \in \mathcal{E}_i$ such that

$$\begin{cases} \mathbf{V}[0] \oplus \mathbf{V}'[0] \oplus \mathbf{x} \neq 0 \\ \text{VOLEHash}(\text{chall}_1, \mathbf{V}[0] \oplus \mathbf{V}'[0] \oplus \mathbf{x}) = 0 \end{cases} \quad (6)$$

for some $\mathbf{x} \in \{0, \mathbf{u} \oplus \mathbf{u}', \mathbf{V}[1] \oplus \mathbf{V}'[1], \dots, \mathbf{V}[k-1] \oplus \mathbf{V}'[k-1]\}$. We now split two cases.

CASE 1. If chall_1 is lastly chosen by H_2^1 query, (6) happens with probability at most

$$Q_{2,1} \cdot \sum_{i \in [\tau]} \binom{|\mathcal{E}_i|}{2} (k+1) \varepsilon_v \leq Q_{2,1} \tau \binom{u}{2} (k+1) \varepsilon_v.$$

CASE 2. Let the above event happened by the $\text{PRG}_{i,j}(\text{seed}, \text{iv})$ query. As $\mathbf{V}[0] \oplus \mathbf{V}'[0] \oplus \mathbf{x}$ is summation of PRG outputs, and VOLEHash is linearly homomorphic, one should satisfy

$$\begin{cases} E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, 0]) = \overline{\text{com}}_{i,j} \\ \text{VOLEHash}(\text{chall}_1, \text{PRG}_{i,j}(\text{seed}, \text{iv})) = C \end{cases}$$

where C is sum of hashed PRG outputs that consists of $\mathbf{V}[0] \oplus \mathbf{V}'[0] \oplus \mathbf{x}$, except $\text{PRG}_{i,j}(\text{seed}, \text{iv})$. At this point, we claim the following, where the proof will be given in Supplementary Material B.2.

Claim 1 *Fix $\text{chall}_1 \in \{0, 1\}^{5\lambda+64}$, $\text{iv} \in \{0, 1\}^{\lambda-32}$, $\overline{\text{com}} \in \{0, 1\}^\lambda$, $C \in \{0, 1\}^{\lambda+B}$. Then, for a fresh PRG query $\text{PRG}_{i,j}(\text{seed}, \text{iv})$, one have*

$$\Pr \left[\begin{array}{l} E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, 0]) = \overline{\text{com}} \\ \text{VOLEHash}(\text{chall}_1, \text{PRG}_{i,j}(\text{seed}, \text{iv})) = C \end{array} \right] \leq \frac{8}{2^{2\lambda+B}}.$$

Then, for each PRG query $\text{PRG}_{i,j}(\text{seed}, \text{iv})$, (6) happens by the query with probability at most

$$\frac{8(k+1)|\mathcal{E}_i|^2 Q_{2,1}}{2^{2\lambda+B}}$$

and as there are at most P of PRG queries, (6) happens with Case 2 with

$$\frac{8(k+1)u^2 Q_{2,1} P}{2^{2\lambda+B}}.$$

All in all, we have

$$\Pr[\mathcal{G}_3 \text{ aborts}] \leq Q_{2,1} \tau \binom{u}{2} (k+1) \varepsilon_v + \frac{8(k+1)u^2 Q_{2,1} P}{2^{2\lambda+B}} \quad (7)$$

Now, we claim that Ext does not return $\perp_{\text{abort}}$ in Step 6. The proof can be found in Supplementary Material B.2.

Claim 2 *Ext does not return $\perp_{\text{abort}}$ in Step 6 unless $\mathcal{G}_3$ aborts.*

$\mathcal{G}_4$: In this game, we will capture the failure condition that Ext returns $\perp_{\text{abort}}$ in Step 7. $\mathcal{G}_4$ checks whether there exists a pair of base VOLEs from different tree that occurs bad consistency check for given challenge chall_1 and corrections, and the game aborts if that happens. In detail, if there exists distinct $i, j \in [\tau]$, $(\mathbf{u}_i, \mathbf{V}_i) \in \mathcal{E}_i$, and $(\mathbf{u}_j, \mathbf{V}_j) \in \mathcal{E}_j$ such that

$$\begin{cases} \mathbf{u}_i \oplus \mathbf{c}_i \neq \mathbf{u}_j \oplus \mathbf{c}_j & (\text{with } \mathbf{c}_0 := 0) \\ \text{VOLEHash}(\text{chall}_1, \mathbf{u}_i \oplus \mathbf{c}_i) = \text{VOLEHash}(\text{chall}_1, \mathbf{u}_j \oplus \mathbf{c}_j), \end{cases}$$

the game aborts.

CASE 1. Suppose that chall_1 is chosen lastly. Then, by the ε_v -universality of VOLEHash and a union bound, the above event happens with probability at most

$$Q_{2,1} \binom{\tau}{2} u^2 \varepsilon_v.$$

CASE 2. Suppose there exists a last PRG query which makes the above event. Then, similarly to the Case 2 of $\mathcal{G}_{3,1}$, the above event happens with probability at most

$$\frac{8(\tau-1)u^2 Q_{2,1} P}{2^{2\lambda+B}}.$$

All in all, we get

$$\Pr[\mathbf{G}_4 \text{ aborts}] \leq Q_{2,1} \binom{\tau}{2} u^2 \varepsilon_v + \frac{8(\tau-1)u^2 Q_{2,1} P}{2^{2\lambda+B}}.$$

Ext does not return $\perp_{\text{abort}}$ in Step 7 unless $\mathbf{G}_4$ aborts, since both $\mathbf{u}_i$ and $\mathbf{u}_j$ should have the same $\tilde{\mathbf{u}}$.

$\mathbf{G}_5$: This game aborts when $\mathcal{A}$ succeed to find a new base VOLE pair $(\hat{\mathbf{u}}, \hat{\mathbf{V}})$ which is consistent to the VOLE commitments and passes the VOLE consistency check after the extraction. In detail, this game aborts when $\mathcal{A}$ makes a fresh PRG query $\text{PRG}_{i,\Delta}(\text{seed}, \text{iv})$ where

- $E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, 0]) = \overline{\text{com}}_{i,\Delta}$;
- there exists a new base VOLE pair $(\hat{\mathbf{u}}, \hat{\mathbf{V}})$ that includes $\text{PRG}_{i,\Delta}(\text{seed}, \text{iv})$ as a VOLE share;
- $(\hat{\mathbf{u}}, \hat{\mathbf{V}})$ passes the consistency check, i.e.,

$$(\text{VOLEHash}(\text{chall}_1, \hat{\mathbf{u}} \oplus \mathbf{c}_i) \oplus \tilde{\mathbf{u}})(\Delta \oplus 1) = \text{VOLEHash}(\text{chall}_1, \hat{\mathbf{V}}) \oplus \tilde{\mathbf{V}}.$$

We remark that $(\hat{\mathbf{u}}, \hat{\mathbf{V}})$ cannot have unopened index other than Δ or $\Delta \oplus 1$ by $\neg \text{BadE}_1$ in the proof of Lemma 1. Moreover, if Δ is the unopened index, then $(\hat{\mathbf{u}}, \hat{\mathbf{V}})$ already existed before this PRG query. So, only $\Delta \oplus 1$ can be the unopened index of $(\hat{\mathbf{u}}, \hat{\mathbf{V}})$.

Then, by applying Claim 1, we have

$$\Pr \left[\begin{array}{l} E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, \Delta \oplus 1, 0]) = \overline{\text{com}}_{i,\Delta \oplus 1} \\ (\text{VOLEHash}(\text{chall}_1, \hat{\mathbf{u}} \oplus \mathbf{c}_i) \oplus \tilde{\mathbf{u}})(\Delta \oplus 1) = \text{VOLEHash}(\text{chall}_1, \hat{\mathbf{V}}) \oplus \tilde{\mathbf{V}} \end{array} \right] \leq \frac{8}{2^{2\lambda+B}}$$

and as there are at most u of $(\hat{\mathbf{u}}, \hat{\mathbf{V}})$, we have

$$\Pr[\mathbf{G}_5 \text{ aborts}] \leq \frac{8uP}{2^{2\lambda+B}}.$$

By this modification, $(\mathbf{u}^*, \mathbf{V}^*)$ is still the unique VOLE extraction that passes consistency check, even we include ideal cipher queries done after the extraction.

From $\mathbf{G}_1$ to $\mathbf{G}_4$, the abort conditions are derived from $\mathcal{L}$, not derived from the adversary's commitment $(\text{chall}_1, \tilde{\mathbf{u}}, h_V)$. Let Γ_{Bad} be the space of $\mathcal{L}$ which leads one of $(\mathbf{G}_i)_{i=1,2,3,4}$ to abort. Then, we have

$$\Pr[\mathcal{L} \in \Gamma_{\text{Bad}}] \leq \frac{Q^2}{2^{2\lambda}} + \frac{18P}{2^\lambda} + \frac{\tau u^2 \varepsilon_v (k + \tau + 1) Q_{2,1}}{2} + \frac{8u^2 (k + \tau) Q_{2,1} P}{2^{2\lambda+B}}.$$

and

$$\Pr[\mathcal{A} \text{ wins} \mid \mathcal{L} \notin \Gamma_{\text{Bad}}] \leq \frac{8uP}{2^{2\lambda+B}}$$

which concludes the proof. $\square$

5.2 Security of rVOLEitH

From the extractable binding property of ChosenVOLECommit, we can prove the EUF-KO security of rVOLEitH. As there is the unique extracted VOLE pair similarly to FAEST, the remaining part – handling ZKHash failure and Quicksilver check failure – of the EUF-KO security proof is similar to that of FAEST. The detailed security proof can be found in Supplementary Material C. Following theorem shows the EUF-KO security of rVOLEitH.

Theorem 1 (EUF-KO Security of rVOLEitH). *Let param be parameters of rVOLEitH. Assume that H_1, H_2^j for $j \in \{1, 2, 3\}$, H_3 , and H_4 are modeled as random oracles. Let $\mathcal{A}$ be an arbitrary adversary against the EUF-KO security of rVOLEitH that makes a total of Q random oracle queries and P ideal cipher queries. Let $Q_1, Q_{2,j}, Q_3$, and Q_4 be the number of queries to H_1, H_2^j, H_3 , and H_4 made by $\mathcal{A}$. Then there exists PPT adversary $\mathcal{B}$ against OWF security of KeyGen such that*

$$\begin{aligned} \text{Adv}_{\text{rVOLEitH}}^{\text{euf-ko}}(\mathcal{A}) &\leq Q_{2,2} \left(\varepsilon_{\text{zk}} + \frac{8uP}{2^{2\lambda+B}} \right) + \frac{2Q_{2,3}}{2^\lambda} + \text{Adv}_{\text{KeyGen}}^{\text{owf}}(\mathcal{B}) \\ &\quad + \frac{Q^2}{2^{2\lambda}} + \frac{18P}{2^\lambda} + \frac{\tau u^2 \varepsilon_v (k + \tau + 1) Q_{2,1}}{2} + \frac{8u^2 (k + \tau) Q_{2,1} P}{2^{2\lambda+B}} \end{aligned}$$

where $u = 2N(\lambda/\log \lambda)^2$, and $\ell_{\text{blk}} P \leq 2^{\lambda-1}$.

The EUF-CMA security is proved by similar game hopping technique as in the FAEST scheme [4], while replacing chall_1 becomes easier as an iv size of 2λ is already sufficient to prevent collision. The major difference from FAEST is the multi-instance hiding game, which is the security notion of IC-VSC rather than VOLECommit. The full proof is provided in Supplementary Material C.

Theorem 2 (EUF-CMA Security of rVOLEitH). *Let param be parameters of rVOLEitH. Assume that $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$, H_2^1, H_2^2 , and H_2^3 are modeled as random oracles. For a PPT adversary $\mathcal{A}$ against the EUF-CMA security of rVOLEitH with a total of Q_{sig} signing oracle queries, Q random oracle queries, and P ideal cipher queries, there exist PPT adversaries*

- $\mathcal{B}$ against the EUF-KO security of rVOLEitH,⁴
- $\mathcal{C}$ against the PRF security of H_3 ,⁵
- $\mathcal{D}$ against the multi-instance hiding security of IC-VSC

such that

$$\begin{aligned} \text{Adv}_{\text{rVOLEitH}}^{\text{euf-cma}}(\mathcal{A}) &\leq \frac{Q_{\text{sig}}(Q_{\text{sig}} + Q)}{2^{2\lambda}} + Q_{\text{sig}} \cdot \text{Adv}_{H_3}^{\text{prf}}(\mathcal{C}) \\ &\quad + \text{Adv}_{\text{IC-VSC}}^{\tau Q_{\text{sig}}\text{-mih}}(\mathcal{D}) + \text{Adv}_{\text{rVOLEitH}}^{\text{euf-ko}}(\mathcal{B}). \end{aligned}$$

⁴ We assume $\mathcal{B}$ has the same amount of queries to random oracles.

⁵ H_3 itself is not a PRF, but it is used as a PRF with key prepending. We use this notation for convenience.

6 Attack on VOLEitH-SBC

In this section, we overview the VOLEitH-SBC signature scheme and provide a practical attack on the scheme. To make the attack more readable, we follow the original notation from [16] which are completely different from the previous sections in this paper.

6.1 Overview of VOLEitH-SBC

Subfield Bilinear Collision (SBC) Problem. Let q be a prime power, and let k, n be positive integers. Given two vectors $\vec{u}, \vec{v} \in (\mathbb{F}_{q^k})^n$ that are linearly independent over $\mathbb{F}_{q^k}$, the goal of the SBC problem is to find two vectors $\vec{x}, \vec{y} \in (\mathbb{F}_q)^n$, linearly independent over $\mathbb{F}_q$, satisfying the relation:

$$(\vec{u} \cdot \vec{x})(\vec{v} \cdot \vec{y}) = (\vec{u} \cdot \vec{y})(\vec{v} \cdot \vec{x}). \quad (8)$$

We denote the instance of the SBC problem as $\text{SBC}[\vec{u}, \vec{v}]$, and if $\vec{x}, \vec{y}$ satisfy the equation above, we write $(\vec{x}, \vec{y}) \in \text{SBC}[\vec{u}, \vec{v}]$.

In the signature scheme based on the SBC problem, a normalized version is used. Instead of arbitrary vectors $\vec{x}, \vec{y}$, solutions should be represented in a form: $\vec{x} = (\vec{x}', 1, 0), \vec{y} = (\vec{y}', 0, 1)$. This results in the normalized SBC (NSBC) problem, where given $\vec{u}, \vec{v} \in (\mathbb{F}_{q^k})^n$, the goal is to find $\vec{x}, \vec{y}$ satisfying the same bilinear relation but in this normalized form.

The hardness of the SBC problem depends on the relation between the parameters k and n . [15] claimed that $k \approx 2n$ is a good choice generally, but they only gives the parameter set $n = 130, k = 257$, and $q = 2$ for 128-bit security. Our attack will also focus on this set of parameters.

Key Generation. To generate an instance of the NSBC problem with a solution, we start by randomly choosing two vectors $\vec{x}, \vec{y} \in (\mathbb{F}_q)^{n-2}$ and setting $\vec{x} = (\vec{x}', 1, 0)$ and $\vec{y} = (\vec{y}', 0, 1)$. Then, we randomly select $\vec{u} = (u_1, \dots, u_n) \in (\mathbb{F}_{q^k})^n$ and $n - 1$ components of $\vec{v} = (v_1, \dots, v_n) \in (\mathbb{F}_{q^k})^n$. The final component of $\vec{v}$, denoted as v_n , is computed to satisfy the NSBC relation:

$$v_n = \frac{(\vec{u} \cdot \vec{y}) \sum_{i=1}^{n-1} v_i x_i - (\vec{u} \cdot \vec{x}) \sum_{i=1}^{n-1} v_i y_i}{y_n (\vec{u} \cdot \vec{x})}. \quad (9)$$

If the denominator happens to be zero, the selection process is restarted. The secret key is $(\vec{x}, \vec{y})$, and the public key is $(\vec{u}, \vec{v})$.

Signing Algorithm. We will briefly describe how the signing algorithm works. Figure 3 in Supplementary Material E is a pseudo-code for the signing algorithm, which is almost same as the pseudo-code in the original paper.

1. (Commitment) Compute cGGM trees and offsets. Each cGGM tree for the VOLE correlation – we will call it *main trees* – is not generated independently, but generated from a single pre-tree. The pre-tree is also a cGGM

tree of 2^τ leaves. By the hypercube technique, pre-tree outputs τ pairs of roots whose sum is always $\vec{x}$. We remark that each main tree is not generated from a single root, but generated from two nodes. So, $\text{cGGM}(x_0, x_1, 2^D)$ outputs a cGGM tree of 2^D leaf nodes whose highest two nodes are x_0 and x_1 . The Folding function for the hypercube technique [1] is defined as follows.

$$\text{Folding}(\vec{x}^{[0]}, \dots, \vec{x}^{[2^\tau-1]}) := \left(\sum_{B_j(i)=0} \vec{x}^{[i]}, \sum_{B_j(i)=1} \vec{x}^{[i]} \right)_{j \in [\tau]}$$

where $B_j(i)$ is j -th bit of i .

For the generation of main trees, VOLE shares are generated from a similar operations. Finally, the offsets $\delta_{\vec{y}_j}, \delta_{z_j}$ are computed. We note that $(\vec{x}^{[0]}, \dots, \vec{x}^{[2^D-1]})$ in Figure 3 plays the role of individual seeds in the original VOLEitH, and there is no commitment to those values.

2. (First Challenge) Hash the transcript and generate the challenges. The challenge includes $\vec{\mu} \in \mathbb{F}_{2^k}^{n-2}$ and $(\alpha_m)_{m \in [\tau D]} \in \mathbb{F}_{2^k}^{\tau D}$.
3. (First Response) Compute the response values $(A, B, (a^{(j)})_{j \in [\tau]}, b)$. A and B are computed as follows. Denoting $\vec{A}^{10} = (\vec{A}, 1, 0)$ and $\vec{A}^{01} = (\vec{A}, 0, 1)$,

$$\begin{aligned} A &\leftarrow (\vec{u} \cdot \vec{A}_{F_x}^{10}) (\vec{v} \cdot \vec{A}_{F_y}^{01}) - (\vec{u} \cdot \vec{A}_{F_y}^{01}) (\vec{v} \cdot \vec{A}_{F_x}^{10}) \\ B &\leftarrow (\vec{v} \cdot \vec{A}_{F_y}^{01}) (\vec{u} \cdot \vec{x}^{10}) + (\vec{u} \cdot \vec{A}_{F_x}^{10}) (\vec{v} \cdot \vec{y}^{01}) \\ &\quad - (\vec{u} \cdot \vec{A}_{F_y}^{01}) (\vec{v} \cdot \vec{x}^{10}) - (\vec{v} \cdot \vec{A}_{F_x}^{10}) (\vec{u} \cdot \vec{y}^{01}) \end{aligned} \quad (10)$$

where $\vec{A}_{F_x}$ and $\vec{A}_{F_y}$ are some linear combinations of $(\alpha_i)_{i \in [\tau D]}$ such that

$$\vec{A}_{F_x} = \sum_{j=0}^{\tau-1} \sum_{i=jD}^{(j+1)D-1} x_j^{[0]i} \alpha_i, \quad \vec{A}_{F_y} = \sum_{j=0}^{\tau-1} \sum_{i=jD}^{(j+1)D-1} y_j^{[0]i} \alpha_i.$$

4. (Second Challenge) Hash the transcript and generate the second challenge $(i_j^*)_{j \in [\tau]}$. Extract pre-challenge i_p^* for pre-tree from the second challenge.
5. (Second Response) Open the puncturable pseudorandom function (PPRF) key. As the highest nodes of the main trees were generated from the pre-tree, the opened PPRF key should contain the PPRF key of both pre-tree and main trees.

Verification Algorithm. Before introducing the verification algorithm, we remark some notations which help to read the attack in the next section. We – and also the original paper [15] – use notation $(\cdot)$ to distinguish the same variables in signing and verify algorithms. For example, $a^{(j)}$ and $a^{(j)'}$, A and A' should be same if the signature was valid. Otherwise, since they computed in different ways, they may be differ from each other.

We will also briefly describe how the verification algorithm works. Figure 4 in Supplementary Material E is a pseudo-code for the verification algorithm, which is exactly same as the pseudo-code in the original paper.

1. (Reconstruct Trees) Parse the signature, reconstruct all the trees in a punctured state, and fold all the trees using the hypercube technique.
2. (First Challenge) Generate the first challenge as in the signing algorithm.
3. (Recompute First Response) Compute $(a^{(j)'})_{j \in [\tau]}$ and A' which should be same as $(a^{(j)})_{j \in [\tau]}$ and A respectively for honest signature.
4. (Compare Second Challenge) Hash the recomputed transcript and compare the hash output.

6.2 Key-Only Attack

We introduce an attack on VOLEitH-SBC signature scheme here. This attack assumes no signature oracle query. Intuitively speaking, the attack manipulates the PPRF key of main trees for $(a^{(j)})_{j \in [\tau]}$ and A to coincide with $(a^{(j)'})_{j \in [\tau]}$ and A' respectively. As the attack seems quite complex, we remark some attack idea before introducing the attack itself.

1. First of all, to forge a signature, the attacker must make the last hash h_1 identical to h'_1 . It implies that, without a hash collision, the last input to the hash function to be matched. That is,

$$(h_0, \text{msg}, A, B, (a^{(j)})_{j \in [\tau]}, b) = (h'_0, \text{msg}, A', B, (a^{(j)'})_{j \in [\tau]}, b).$$

We note that (msg, B, b) is always same on the both sides.

2. The attacker must also make the hash value h_0 identical to h'_0 . It also implies that, without a hash collision, the input to the hash function to be matched. If $(\text{msg}, (\delta_{y_j}, \delta_{z_j})_{j \in [\tau]})$ is fixed, then they are always same.
3. The attacker must also make $(a^{(j)})_{j \in [\tau]}$ identical to $(a^{(j)'})_{j \in [\tau]}$. Since $a^{(j)'}$ is a linear combination of $y_j^{\llbracket b_{i,j} \rrbracket_i}$ and $z_j^{\llbracket b_{i,j} \rrbracket_i}$, if there are sufficiently many $y_j^{\llbracket b_{i,j} \rrbracket_i}$ and $z_j^{\llbracket b_{i,j} \rrbracket_i}$, then the attacker can make them identical by using Wagner's k -XOR algorithm [22]. Since there is no leaf commitment, the GGM trees can be freely manipulated by the attacker. Therefore, sufficiently many $y_j^{\llbracket b_{i,j} \rrbracket_i}$ and $z_j^{\llbracket b_{i,j} \rrbracket_i}$ can be generated from the GGM trees.
4. To make A and A' coincide, the attacker should make $A = \Phi(\Delta^*) - B\Delta^*$. If $h_0 = h'_0$ and B are fixed, the equality holds with probability about $2^{-\lambda}$ since Δ^* is a linear combination of α_m 's which are generated from $\mathcal{H}(m, h_0)$. Since $\Phi(\Delta^*)$ is manipulatable after fixing Δ^* , the attacker will make a collision by making $2^{\lambda/2}$ of Δ^* and $2^{\lambda/2}$ of $\Phi(\Delta^*)$. Then, this makes the forgery.

Because of the page limitation, we provide the detailed attack algorithm in Supplementary Material D. For all the parameter sets given in [16] which ranges from $(D, \tau) = (9, 15)$ to $(15, 9)$, the attack costs are similar and upper bounded by 2^{69} up to some constant multiple.

References

1. Aguilar-Melchor, C., Gama, N., Howe, J., Hülsing, A., Joseph, D., Yue, D.: The return of the SDitH. In: EUROCRYPT 2023. pp. 564–596. Springer (2023)
2. Baum, C., Beullens, W., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: One Tree to Rule Them All: Optimizing GGM Trees and OWFs for Post-Quantum Signatures. Cryptology ePrint Archive, Paper 2024/490 (2024), <https://eprint.iacr.org/2024/490>
3. Baum, C., Braun, L., Munch-Hansen, A., Scholl, P.: Moz $\mathbb{Z}_{2^k}$ arella: Efficient Vector-OLE and Zero-Knowledge Proofs Over $\mathbb{Z}_{2^k}$. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology – CRYPTO 2022. pp. 329–358. Springer Nature Switzerland, Cham (2022)
4. Baum, C., Braun, L., de Saint Guilhem, C.D., Klooß, M., Majenz, C., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: FAEST. Technical report, National Institute of Standards and Technology, 2023 (2023), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>
5. Baum, C., Braun, L., de Saint Guilhem, C.D., Klooß, M., Orsini, E., Roy, L., Scholl, P.: Publicly Verifiable Zero-Knowledge and Post-Quantum Signatures from VOLE-in-the-Head. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023. pp. 581–615. Springer Nature Switzerland, Cham (2023)
6. Bidoux, L., Feneuil, T., Gaborit, P., Neveu, R., Rivain, M.: Dual support decomposition in the head: Shorter signatures from rank sd and minrank. In: Chung, K.M., Sasaki, Y. (eds.) Advances in Cryptology – ASIACRYPT 2024. pp. 38–69. Springer Nature Singapore, Singapore (2025)
7. Bienstock, A., Patel, S., Seo, J.Y., Yeo, K.: Near-Optimal Oblivious Key-Value Stores for Efficient PSI, PSU and Volume-Hiding Multi-Maps. In: 32nd USENIX Security Symposium (USENIX Security 23). pp. 301–318. USENIX Association, Anaheim, CA (Aug 2023), <https://www.usenix.org/conference/usenixsecurity23/presentation/bienstock>
8. Bui, D.: Shorter VOLEitH Signature from Multivariate Quadratic. Cryptology ePrint Archive (2024)
9. Bui, D., Carozza, E., Couteau, G., Goudarzi, D., Joux, A.: Short Signatures from Regular Syndrome Decoding, Revisited. Cryptology ePrint Archive, Paper 2024/252 (2024), <https://eprint.iacr.org/2024/252>
10. Bui, D., Cong, K., de Saint Guilhem, C.D.: Improved all-but-one vector commitment with applications to post-quantum signatures. Cryptology ePrint Archive, Paper 2024/097 (2024), <https://eprint.iacr.org/2024/097>
11. Cui, H., Liu, H., Yan, D., Yang, K., Yu, Y., Zhang, K.: ResolveD: Shorter signatures from regular syndrome decoding and vole-in-the-head. In: PKC 2024. pp. 229–258. Springer (2024)
12. Dobraunig, C., Kales, D., Rechberger, C., Schofnegger, M., Zaverucha, G.: Shorter Signatures Based on Tailor-Made Minimalist Symmetric-Key Crypto. In: ACM CCS 2022. pp. 843–857. Association of Computing Machinery (November 2022), <https://www.microsoft.com/en-us/research/publication/shorter-signatures-based-on-tailor-made-minimalist-symmetric-key-crypto/>
13. Goldreich, O., Goldwasser, S., Micali, S.: How to Construct Random Functions. J. ACM **33**(4), 792–807 (aug 1986). <https://doi.org/10.1145/6490.6503>, <https://doi.org/10.1145/6490.6503>

14. Guo, X., Yang, K., Wang, X., Zhang, W., Xie, X., Zhang, J., Liu, Z.: Half-tree: Halving the cost of tree expansion in cot and dpf. In: EUROCRYPT 2023. pp. 330–362. Springer (2023)
15. Huth, J., Joux, A.: Mpc in the head using the subfield bilinear collision problem. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology – CRYPTO 2024*. pp. 39–70. Springer Nature Switzerland, Cham (2024)
16. Huth, J., Joux, A.: VOLE-in-the-head signatures from subfield bilinear collisions. *Cryptology ePrint Archive*, Paper 2024/1537 (2024), <https://eprint.iacr.org/2024/1537>
17. Ishai, Y., Kushilevitz, E., Ostrovsky, R., Sahai, A.: Zero-knowledge from Secure Multiparty Computation. In: *ACM STOC 2007*. pp. 21–30 (2007)
18. Kim, S., Ha, J., Son, M., Lee, B., Moon, D., Lee, J., Lee, S., Kwon, J., Cho, J., Yoon, H., Lee, J.: Aim: Symmetric primitive for shorter signatures with stronger security. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. p. 401–415. CCS ’23, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3576915.3616579>, <https://doi.org/10.1145/3576915.3616579>
19. Kim, S., Lee, B., Son, M.: Relaxed Vector Commitment for Shorter Signatures. *Cryptology ePrint Archive*, Paper 2024/1004 (2024), <https://eprint.iacr.org/2024/1004>
20. Rindal, P., Schoppmann, P.: VOLE-PSI: Fast OPRF and Circuit-PSI from Vector-OLE. In: *EUROCRYPT 2021*. pp. 901–930. Springer, Cham (2021)
21. Roy, L.: SoftSpokenOT: Communication–Computation Tradeoffs in OT Extension. *Cryptology ePrint Archive* (2022)
22. Wagner, D.: A Generalized Birthday Problem. In: Yung, M. (ed.) *Advances in Cryptology — CRYPTO 2002*. pp. 288–304. Springer Berlin Heidelberg, Berlin, Heidelberg (2002)
23. Weng, C., Yang, K., Katz, J., Wang, X.: Wolverine: Fast, Scalable, and Communication-Efficient Zero-Knowledge Proofs for Boolean and Arithmetic Circuits. In: *2021 IEEE Symposium on Security and Privacy (S&P)*. pp. 1074–1091 (2021). <https://doi.org/10.1109/SP40001.2021.00056>
24. Yang, K., Sarkar, P., Weng, C., Wang, X.: QuickSilver: Efficient and Affordable Zero-Knowledge Proofs for Circuits and Polynomials over Any Field. In: *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. p. 2986–3001. CCS ’21, Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3460120.3484556>, <https://doi.org/10.1145/3460120.3484556>
25. Zaverucha, G., Chase, M., Derler, D., Goldfeder, S., Orlandi, C., Ramacher, S., Rechberger, C., Slamanig, D., Katz, J., Wang, X., Kolesnikov, V., Kales, D.: Picnic. Technical report, National Institute of Standards and Technology, 2020 (2022), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions>

Supplementary Material

A Complete Set of Algorithms in rVOLEitH

A.1 Sign and Verify

The signing and verification algorithms for rVOLEitH are presented in Algorithm 6 and Algorithm 7, respectively. In the **Sign** algorithm, we do not explicitly define the function **ExtendWitness**, which extracts the witness of the one-way function. At a high-level, starting from the secret key **sk**, the output of each multiplication gate within the one-way function is treated as a witness. To aid understanding, the function **ExtendWitness** for AES is formally specified as **FAEST.AES.ExtendWitness** in FAEST [4].

A.2 Unchanged Algorithms

The functions **VOLEHash**, **ZKHash**, **ChalDec**, **ToField**, and **ToBits** remain unchanged from the FAEST specification. The functions **OWFProve** and **OWFVerify** extend **FAEST.AES.AESProve** and **FAEST.AES.AESVerify** in FAEST to support arbitrary circuits. Apart from this generalization, the computation remains the same.

Algorithm 6. $\text{Sign}(\text{msg}, \text{sk}, \text{pk}; \text{param}, \text{param}_{\text{OWF}})$ - rVOLEitH, signing algorithm.

```

// Randomness generation
1  $\rho \leftarrow_{\$} \{0, 1\}^\lambda$  ( $\rho \leftarrow 0^\lambda$  for deterministic signature)
2  $\mu \leftarrow H_1(\text{pk} \parallel \text{msg})$ 
3  $\text{iv} \in \{0, 1\}^{2\lambda} \leftarrow H_3(\text{sk} \parallel \mu \parallel \rho)$ 
// Commitment and VOLE derandomization
4  $\mathbf{w} \leftarrow \text{ExtendWitness}(\text{sk}, \text{pk}; \text{param}, \text{param}_{\text{OWF}})$ 
5  $(h_{\text{com}}, \text{decom}_0, \dots, \text{decom}_{\tau-1}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \mathbf{u}, \mathbf{V}) \leftarrow \text{VOLECommit}(\mathbf{w}[:\lambda], \text{iv}, \ell + 2\lambda + B; \text{param})$ 
6  $\text{chall}_1 \leftarrow H_2^1(\mu \parallel h_{\text{com}} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}; 5\lambda + 64)$ 
// VOLE consistency check and witness commitment
7  $\tilde{\mathbf{u}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{u})$ 
8  $\tilde{\mathbf{V}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{V})$ 
9  $h_V \leftarrow H_1(\tilde{\mathbf{V}})$ 
10  $\mathbf{d} \leftarrow \mathbf{w}[\lambda : \ell] \oplus \mathbf{u}[\lambda : \ell]$ 
11  $\text{chall}_2 \leftarrow H_2^2(\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel h_V \parallel \mathbf{d}; 3\lambda + 64)$ 
// Proof of knowledge of OWF witness
12  $\mathbf{u} \leftarrow \mathbf{u}[:\ell + \lambda]$ 
13  $\mathbf{V} \leftarrow \mathbf{V}[:\ell + \lambda]$ 
14  $(\tilde{a}, \tilde{b}) \leftarrow \text{OWFProve}(\mathbf{w}, \mathbf{u}, \mathbf{V}, \text{pk}, \text{chall}_2; \text{param}, \text{param}_{\text{OWF}})$ 
15  $\text{ctr} \leftarrow 0, \text{reject} \leftarrow 1$ 
16 while reject do
17    $\text{chall}_3 \leftarrow H_2^3(\text{chall}_2 \parallel \tilde{a} \parallel \tilde{b} \parallel \text{ctr}; \lambda)$ 
18   if  $\text{chall}_3[\lambda - w : \lambda] \neq 0^w$  then
19      $\text{ctr} \leftarrow \text{ctr} + 1$ 
20   else
21      $\text{reject} \leftarrow 0$ 
22 for  $i \in [\tau]$  do
23    $\mathbf{s}_i \leftarrow \text{ChalDec}(\text{chall}_3, i; \text{param})$ 
24    $\text{pdecom}_i \leftarrow \text{IC-VSC.Open}(\text{decom}_i, \mathbf{s}_i)$ 
25 return  $\sigma := ((\mathbf{c}_i)_{i \in [1, \tau]}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}, (\text{pdecom}_i)_{i \in [\tau]}, \text{chall}_3, \text{iv}, \text{ctr})$ 

```

Algorithm 7. $\text{Verify}(\text{msg}, \text{pk}; \text{param}, \text{param}_{\text{OWF}})$ - rVOLEitH, verification algorithm.

```

// Randomness generation
1 Parse  $\sigma = ((\mathbf{c}_i)_{i \in [1, \tau]}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}, (\text{pdecom}_i)_{i \in [\tau]}, \text{chall}_3, \text{iv}, \text{ctr})$ 
2  $\mu \leftarrow H_1(\text{pk} \parallel \text{msg})$ 
// Reconstruct VOLEs and check commitment
3  $(h_{\text{com}}, \mathbf{Q}'_0, \dots, \mathbf{Q}'_{\tau-1}) \leftarrow$ 
    $\text{VOLEReconstruct}(\text{chall}_3, (\text{pdecom}_i)_{i \in [\tau]}, \text{iv}, \ell + 2\lambda + B; \text{param}).$ 
4  $\text{chall}_1 \leftarrow H_2^1(\mu \parallel h_{\text{com}} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}; 5\lambda + 64)$ 
// Apply the VOLE correction values and check consistency
5  $\mathbf{Q}_0 \leftarrow \mathbf{Q}'_0$ 
6 for  $i \in [\tau]$  do
7   if  $i \leq \tau_0$  then  $b \leftarrow 0$  else  $b \leftarrow 1$ 
8    $(\delta_0, \dots, \delta_{k_b-1}) \leftarrow \text{ChalDec}(\text{chall}_3, i; \text{param})$ 
9    $\tilde{\mathbf{D}}_i \leftarrow [\delta_0 \cdot \tilde{\mathbf{u}} \dots \delta_{k_b-1} \cdot \tilde{\mathbf{u}}] \in \{0, 1\}^{(\lambda+B) \times k_b}$ 
10   $\mathbf{c}_i \leftarrow 0^\lambda \parallel \mathbf{c}_i$ 
11  if  $i \geq 1$  then
12     $\mathbf{Q}_i \leftarrow \mathbf{Q}_0 \oplus [\delta_0 \cdot \mathbf{c}_i \dots \delta_{k_b} \cdot \mathbf{c}_i]$ 
13 Denote  $\mathbf{Q} = [\mathbf{Q}_0 \dots \mathbf{Q}_{\tau-1}]$ , and  $\tilde{\mathbf{D}} = [\tilde{\mathbf{D}}_0 \dots \tilde{\mathbf{D}}_{\tau-1}]$ 
14  $\tilde{\mathbf{Q}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{Q}) \in \{0, 1\}^{(\lambda+B) \times \lambda}$ 
15  $h_V \leftarrow H_1(\tilde{\mathbf{Q}} \oplus \tilde{\mathbf{D}})$ 
16  $\text{chall}_2 \leftarrow H_2^2(\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel h_V \parallel \mathbf{d}; 3\lambda + 64)$ 
// Compute AES consistency values
17  $\tilde{b} \leftarrow \text{OWFVerify}(0^\lambda \parallel \mathbf{d}, \mathbf{Q}_{[\ell+\lambda]}, \text{chall}_2, \text{chall}_3, \tilde{a}, \text{pk}; \text{param}, \text{param}_{\text{OWF}})$ 
18  $\text{chall}'_3 \leftarrow H_2^3(\text{chall}_2 \parallel \tilde{a} \parallel \tilde{b} \parallel \text{ctr}; \lambda)$ 
19 if  $\text{chall}_3 = \text{chall}'_3$  then
20   return ACCEPT
21 else
22   return REJECT

```

Algorithm 8. $\text{VOLEHash}(\text{sd}, (\mathbf{x}_0, \mathbf{x}_1)) \in \{0, 1\}^{\ell+\lambda} \times \{0, 1\}^{\lambda+B}$

```

1  $\lambda$ -bit  $\mathbf{r}_i$ ,  $\mathbf{s}$ , 64-bit  $\mathbf{t}$ 
2 Parse  $\text{sd} = (\mathbf{r}_0 \parallel \mathbf{r}_1 \parallel \mathbf{r}_2 \parallel \mathbf{r}_3 \parallel \mathbf{s} \parallel \mathbf{t}) \in \{0, 1\}^{5\lambda+64}$ 
3  $r_i \leftarrow \text{ToField}(\mathbf{r}_i, \lambda)$  for  $i \in [4]$ 
4  $s \leftarrow \text{ToField}(\mathbf{s}, \lambda)$ ,  $t \leftarrow \text{ToField}(\mathbf{t}, 64)$ 
5  $\ell' \leftarrow \lambda \cdot \lceil (\ell + \lambda) / \lambda \rceil$ 
6  $\mathbf{x}_0 \leftarrow \mathbf{x}_0 \parallel 0^{\ell' - (\ell + \lambda)}$  // pad to multiple of  $\lambda$ 
7  $\hat{\mathbf{y}} \leftarrow \text{ToField}(\mathbf{x}_0, \lambda)$ ,  $\bar{\mathbf{y}} \leftarrow \text{ToField}(\mathbf{x}_0, 64)$ 
8  $h_0 \leftarrow \sum_{i=0}^{\ell'/\lambda-1} s^{\ell'/\lambda-1-i} \cdot \hat{\mathbf{y}}[i]$  (in  $\mathbb{F}_{2^\lambda}$ )
9  $h_1 \leftarrow \sum_{i=1}^{\ell'/\lambda-1} t^{\ell'/64-1-i} \cdot \bar{\mathbf{y}}[i]$  (in  $\mathbb{F}_{2^{64}}$ )
10  $h'_1 \leftarrow \text{ToField}(\text{ToBits}(h_2) \parallel 0^{\lambda-64}, \lambda)$ 
11  $(h_2, h_3) \leftarrow (r_0 h_0 + r_1 h'_1, r_2 h_0 + r_3 h'_1)$ 
12  $\mathbf{h} \leftarrow (\text{ToBits}(h_2) \parallel (\text{ToBits}(h_3)[B])) \oplus \mathbf{x}_2$ 
13 return  $\mathbf{h}$ 

```

Algorithm 9. $\text{ZKHash}(\text{sd}, (\mathbf{x}_0, x_1)) \in \mathbb{F}_{2^\lambda}^\ell \times \mathbb{F}_{2^\lambda}$

```

1  $\lambda$ -bit  $\mathbf{r}_i$ ,  $\mathbf{s}$ ; 64-bit  $\mathbf{t}$ 
2 Parse  $\text{sd} = (\mathbf{r}_0 \parallel \mathbf{r}_1 \parallel \mathbf{s} \parallel \mathbf{t}) \in \{0, 1\}^{3\lambda+64}$ 
3  $r_i \leftarrow \text{ToField}(\mathbf{r}_i, \lambda)$  for  $i \in [2]$ 
4  $s \leftarrow \text{ToField}(\mathbf{s}, \lambda)$ 
5  $t \leftarrow \text{ToField}(\mathbf{t} \parallel 0^{\lambda-64}, \lambda)$ 
6  $h_1 \leftarrow \sum_{i=0}^{\ell-1} s^{\ell-1-i} \cdot \mathbf{x}_0[i]$  (in  $\mathbb{F}_{2^\lambda}$ )
7  $h_2 \leftarrow \sum_{i=0}^{\ell-1} t^{\ell-1-i} \cdot \mathbf{x}_0[i]$  (in  $\mathbb{F}_{2^\lambda}$ )
8  $\mathbf{h} \leftarrow \text{ToBits}(r_0 h_0 + r_1 h_1 + x_1)$ 
9 return  $\mathbf{h}$ 

```

Algorithm 10. $\text{ToField}(\mathbf{x}, k)$

```

1 let  $\alpha_k \in \mathbb{F}_{2^k}$ 
2 if  $\mathbf{x} \in \{0, 1\}^k$  then
3   return  $y := \sum_{i=0}^{k-1} \mathbf{x}[i] \cdot \alpha_k^i$ 
4 if  $\mathbf{x} \in \{0, 1\}^{nk}$  then
5   new  $\mathbf{y} \in \mathbb{F}_{2^k}^n$ 
6   for  $i \in [n]$  do
7      $\mathbf{y}[i] \leftarrow \sum_{j=0}^{k-1} \mathbf{x}[ni+j] \cdot \alpha_k^j$ 
8   return  $\mathbf{y}$ 
9 else
10  return  $\perp$ 

```

Algorithm 11. ToBits($\mathbf{x}; k, n$)

```

1 new  $\mathbf{y} := ()$ 
2 for  $i \in [n]$  do
3   Parse  $x_0 + x_1\alpha_k + \dots + x_{k-1}\alpha_k^{k-1} = \mathbf{x}[i]$  for  $x_j \in \mathbb{F}_2$ 
4    $\mathbf{y}_i \leftarrow x_0 \parallel \dots \parallel x_{k-1} \in \{0, 1\}^k$ 
5    $\mathbf{y} \leftarrow \mathbf{y} \parallel \mathbf{y}_i$ 
6 return  $\mathbf{y}$ 

```

Algorithm 12. OWFProve($\mathbf{w}, \mathbf{u}, \mathbf{V}, \text{pk}, \text{chall}; \text{param}, \text{param}_{\text{OWF}}$)

```

1 Represent OWF as circuit  $\mathcal{C}$  with  $C$  multiplication gates
2  $j \leftarrow 0$ 
3 for gate  $g \in \mathcal{C}$  do
4   Denote the input wire  $x$  and  $y$ 
5   Denote the output wire  $z$ 
6   if  $g$  is ADD then
7      $\mathbf{w}_z \leftarrow \mathbf{w}_x + \mathbf{w}_y$ 
8      $\mathbf{v}_z \leftarrow \mathbf{v}_x + \mathbf{v}_y$ 
9   if  $g$  is MUL then
10     $A_{1,j} \leftarrow \mathbf{v}_x \cdot \mathbf{v}_y$ 
11     $A_{2,j} \leftarrow (\mathbf{w}_x + \mathbf{v}_x)(\mathbf{w}_y + \mathbf{v}_y) - w_z - A_{1,j}$ 
12     $j \leftarrow j + 1$ 
13  $\mathbf{a}_1 \leftarrow (A_{1,1}, \dots, A_{1,C})$ 
14  $\mathbf{a}_2 \leftarrow (A_{2,1}, \dots, A_{2,C})$ 
15 for  $i \in [\ell, \ell + \lambda]$  do
16    $\mathbf{u}[i - \ell] \leftarrow \text{ToField}(\mathbf{u}[i], \lambda)$ 
17  $u^* \leftarrow \sum_{i \in [\lambda]} \mathbf{u}[i] \alpha_\lambda^i$ 
18  $v^* \leftarrow \sum_{i \in [\lambda]} \mathbf{v}[\ell + i] \alpha_\lambda^i \text{ // } \alpha_\lambda \in \mathbb{F}_{2^\lambda}$ 
19  $\tilde{a} \leftarrow \text{ZKHash}(\text{chall}, \mathbf{a}_2 \parallel u^*)$ 
20  $\tilde{b} \leftarrow \text{ZKHash}(\text{chall}, \mathbf{a}_1 \parallel v^*)$ 
21 return  $(\tilde{a}, \tilde{b})$ 

```

Algorithm 13. OWFVerify($\mathbf{d}, \mathbf{Q}, \text{chall}_2, \text{chall}_3, \tilde{a}, \text{pk}; \text{param}, \text{param}_{\text{OWF}}$)

```

1 Represent OWF as circuit  $\mathcal{C}$  with  $C$  multiplication gates
2  $\Delta \leftarrow \text{ToField}(\text{chall}_3; \lambda)$ 
3 Write  $\mathbf{Q} = [\mathbf{Q}_0, \dots, \mathbf{Q}_{\tau-1}]$ 
4 for  $i \in [\tau]$  do
5   if  $i < \tau_0$  then  $b \leftarrow 0$  else  $b \leftarrow 1$ 
6    $(\delta_0, \dots, \delta_{k_b-1}) \leftarrow \text{ChalDec}(\text{chall}_3, i; \text{param})$ 
7   Write  $\mathbf{Q}_i = [\mathbf{q}_{i,0}, \dots, \mathbf{q}_{i,k_b-1}]$ 
8   for  $j \in [k_b]$  do
9      $\mathbf{q}_{i,j}[:\ell] \leftarrow \mathbf{q}_{i,j}[:\ell] \oplus \delta_j \cdot \mathbf{d}$ 
10 for  $i \in [\ell + \lambda]$  do
11    $\mathbf{q}[i] \leftarrow \text{ToField}(\mathbf{Q}[i]; \lambda)$ 
12 for gate  $g \in C$  do
13   Denote the input wire  $x$  and  $y$ 
14   Denote the output wire  $z$ 
15   if  $g$  is ADD then
16      $\mathbf{q}_z \leftarrow \mathbf{q}_x + \mathbf{q}_y$ 
17   if  $g$  is MUL then
18      $B_j \leftarrow \mathbf{q}_x \mathbf{q}_y - w_z \cdot \Delta^2$ 
19  $\mathbf{b} \leftarrow (B_1, \dots, B_C)$ 
20  $q^* \leftarrow \sum_{i \in [\lambda]} \mathbf{q}[\ell + i] \alpha_\lambda^i // \alpha_\lambda \in \mathbb{F}_{2^\lambda}$ 
21  $\tilde{q} \leftarrow \text{ZKHash}(\text{chall}, \mathbf{b} || q^*)$ 
22 return  $\tilde{q} - \tilde{a} \cdot \Delta$ 

```

Algorithm 14. ChalDec($\text{chall}, i; \text{param}$) - Challenge decomposition algorithm.

```

1 if  $i \notin [\tau_0 + \tau_1]$  then
2   abort
3 Parse  $\text{chall} \in \{0, 1\}^{k_0 \tau_0 + k_1 \tau_1}$ 
4 if  $i < \tau_0$  then
5    $\text{lo} := i \cdot k_0$ 
6    $\text{hi} := (i + 1) \cdot k_0 - 1$ 
7 else
8    $t := i - \tau_0$ 
9    $\text{lo} := \tau_0 k_0 + t \cdot k_1$ 
10   $\text{hi} := \tau_0 k_0 + (t + 1) \cdot k_1 - 1$ 
11 return  $\text{chall}[\text{lo} : \text{hi}]$ 

```

B Proofs of Lemmas

In this section, we provide some proofs of lemmas and claims. Before we provide the proofs, we provide a useful fact and a technical lemma. This fact and lemma was introduced in [19], but we slightly modified some numbers and adapt the proof.

Chernoff bound. Let X be a random variable following the binomial distribution $B(Q, p)$. Then, the probability of the tail of X is upper bounded by a Chernoff bound

$$\Pr[X \geq (1 + \delta)\mu] \leq \left(\frac{e^\delta}{(1 + \delta)^{1+\delta}} \right)^\mu < \frac{1}{e^\mu} \cdot \left(\frac{e\mu}{(1 + \delta)\mu} \right)^{(1+\delta)\mu}$$

for any $\delta > 0$, where e is Euler's number and $\mu = Q \cdot p$ is the expectation of X .

For given a multiset $S = \{x_1, x_2, \dots, x_Q\}$ with $x_i \leftarrow_{\$} \{0, 1\}^\lambda \setminus \mathcal{D}_i$ where $|\mathcal{D}_i| \leq 2^{\lambda-1}$, we can bound the probability that the maximum number of multi-collision mcoll in S is greater than $2\lambda/\log \lambda$ as follows. With $\lambda \geq 64$ and $Q \leq 2^{\lambda-1}$, the probability is bounded by

$$\begin{aligned} \Pr[\text{mcoll} \geq 2\lambda/\log \lambda] &< \left(\frac{e \log \lambda}{2\lambda} \cdot \frac{2Q}{2^\lambda} \right)^{2\lambda/\log \lambda} \cdot 2^\lambda \leq \left(\frac{e \log \lambda}{\lambda^{1/2}} \cdot \frac{Q}{2^\lambda} \right)^{2\lambda/\log \lambda} \\ &\leq \left(\frac{2e}{3} \cdot \frac{Q}{2^\lambda} \right)^{2\lambda/\log \lambda} \leq \frac{2Q}{2^\lambda}. \end{aligned} \quad (11)$$

Similarly, given a multiset $S' = \{y_1, \dots, y_Q\}$ with $y_i \leftarrow_{\$} \{0, 1\}^c \setminus \mathcal{D}'_i$ where $|\mathcal{D}'_i| \leq 2^{c-1}$, we can bound the maximum multi-collision mcoll' in S' is greater than 13 as follows. With $Q^6 \leq 2^{5c}$,

$$\Pr[\text{mcoll}' \geq 13] < \left(\frac{2eQ}{13 \cdot 2^c} \right)^{13} \cdot 2^c \leq \frac{1}{2^{16}} \cdot \frac{Q}{2^{2c}}. \quad (12)$$

H-coefficient Technique. The H-coefficient technique is a method which provide a way to bound the distinguishing advantage of an adversary interacting with an idealized cryptographic system and a real algorithm. The core idea of the H-coefficient technique is to partition the set of possible transcripts (i.e., sequences of queries and responses) into two subsets: “good” and “bad” transcripts. The probability of bad transcripts can be shown to be negligible, while the probability of distinguishing between the distributions of good transcripts can be tightly bounded. The following lemma formally state the H-coefficient technique.

Lemma 4 (H-Coefficient Lemma) *Let $\mathcal{A}$ be an algorithm that interacts with either an ideal world or a real world and tries to distinguish two worlds. Let T_{id} and T_{re} be the distribution of transcript in the ideal world and the real world, respectively, and $\mathcal{T}$ denotes the set of all attainable transcripts in the ideal world.*

Suppose there exist partition $\mathcal{T}_{\text{Good}}$ (good transcripts) and $\mathcal{T}_{\text{Bad}}$ (bad transcripts) of $\mathcal{T}$, and constants ϵ and δ such that for any $\gamma \in \mathcal{T}_{\text{Good}}$,

$$\Pr[T_{\text{id}} \in \mathcal{T}_{\text{Bad}}] \leq \epsilon, \quad \frac{\Pr[\gamma = T_{\text{re}}]}{\Pr[\gamma = T_{\text{id}}]} \geq 1 - \delta.$$

Then, the distinguishing advantage of $\mathcal{A}$ in distinguishing $\mathcal{I}$ from $\mathcal{R}$ is bounded by:

$$\text{Adv}_{\mathcal{I}, \mathcal{R}}^{\text{dist}}(\mathcal{A}) \leq \epsilon + \delta.$$

Commitment Collision Probability. When bounding the number of `pdecom`, the most important fact is that there is no collision on between nodes of positive height, where a collision means that the related leaf commitments are all the same. Lemma 5 states that the probability of the collision is negligible. We note that, in Lemma 5, a collision of truncated IV ($\text{iv}_{\text{pt}}[: \lambda - 32]$) does not affect the probability. Therefore, the proof of the lemma is exactly the same with Lemma 5 in [19] even though the statement is slightly different from it.

Lemma 5 *Let $E : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be an ideal cipher and $\sigma : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be an orthomorphism. Let $\mathcal{A}$ be an arbitrary adversary that makes P queries to E . Then, the probability that $\mathcal{A}$ finds $\text{iv} = (\text{iv}_{\text{pt}}, \text{iv}_{\text{key}}, \mathbf{b}) \in \{0, 1\}^{\lambda + \lambda + 8}$, $i \in \{0, 2, \dots, 2^{16} - 2\}$, and distinct $n, n' \in \{0, 1\}^\lambda$ such that*

$$\begin{cases} E_{\sigma(n) \oplus E_{\text{iv}_{\text{key}}}(n \oplus \text{iv}_{\text{pt}})}(\text{iv}_{\text{pt}}[: \lambda - 32] \parallel \mathbf{b} \parallel \langle i \rangle_{2B} \parallel \langle 0 \rangle_B) \\ = E_{\sigma(n') \oplus E_{\text{iv}_{\text{key}}}(n' \oplus \text{iv}_{\text{pt}})}(\text{iv}_{\text{pt}}[: \lambda - 32] \parallel \mathbf{b} \parallel \langle i \rangle_{2B} \parallel \langle 0 \rangle_B), \\ E_{\sigma(n) \oplus E_{\text{iv}_{\text{key}}}(n \oplus \text{iv}_{\text{pt}}) \oplus n}(\text{iv}_{\text{pt}}[: \lambda - 32] \parallel \mathbf{b} \parallel \langle i + 1 \rangle_{2B} \parallel \langle 0 \rangle_B) \\ = E_{\sigma(n') \oplus E_{\text{iv}_{\text{key}}}(n' \oplus \text{iv}_{\text{pt}}) \oplus n'}(\text{iv}_{\text{pt}}[: \lambda - 32] \parallel \mathbf{b} \parallel \langle i + 1 \rangle_{2B} \parallel \langle 0 \rangle_B), \end{cases} \quad (13)$$

is at most $10P/2^\lambda$.

B.1 Lemmas on IC-VSC

In this section, we provide the proof of Lemma 1 and Lemma 2. The proofs are basically same with those of Lemma 6 and Lemma 7, respectively. Nevertheless, we provide sketches of the proofs to be self-contained.

There are two differences between IC-VSC in this paper and IC-VSC in [18]: truncated last blocks, and 16-bit VOLE share counter $\langle i \rangle_{2B}$. As those differences does not affect the u -ESB security, the proof will be same as that in [19].

Lemma 1 (Extractable Semi-binding, [19]) *Let $E : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be an ideal cipher. Let $\mathcal{A}$ be an arbitrary adversary that makes P queries to E . Then, $\mathcal{A}$'s u -extractable semi-binding advantage $\text{Adv}_{\text{IC-VSC}}^{u\text{-esb}}(\mathcal{A})$ against IC-VSC is bounded by*

$$\text{Adv}_{\text{IC-VSC}}^{u\text{-esb}}(\mathcal{A}) \leq \frac{14P}{2^\lambda},$$

for $u = 2^{k+1} \left(\frac{\lambda}{\log \lambda} \right)^2$.

Proof (Sketch). Let P be the number of queries to E , and $\mathcal{L}_E$ be the collection of all queries to E . Since the choice of **salt** and **b** does not affect extractable semi-binding, we can assume all salts and **b** are equaled to zero strings. For the i -th node $\text{node}_{e,i}$ of depth e , we will say $\text{node}_{e,i}$ is *compatible* with **com** if all the leaf commitments of height-0 descendant nodes are same with **com** in the position from $i \cdot 2^{k-e}$ (inclusive) to $(i+1) \cdot 2^{k-e}$ (exclusive).

We define the extractor $\text{Ext}(0^\lambda, \mathcal{L}_E, \text{com} = (\text{com}_0, \dots, \text{com}_{2^k-1}))$ as follows.

1. For $e \in [k]$ and $i \in [2^e]$, find sets $S_{e,i}$ of the i -th internal nodes of depth e compatible with **com**.
2. For $i \in [2^k]$, construct sets of copath of i -th leaf node A_i where the internal nodes are in $S_{e,i}$.
3. Find set of messages S of $(s_j)_{j \in [2^k]}$ such that $\text{Recon}(a, \text{com}_i, [2^k] \setminus \{i\}) = (s_j)_{j \neq i}$. If $S_{k,i} = \emptyset$ then set $s_i = \perp$, or set s_i as one of elements in $S_{k,i}$ otherwise. Ext outputs arbitrary u elements if $|S| > u$, or outputs all the elements of S otherwise.

Now we define some bad events.

- $\text{BadE}_1 \Leftrightarrow$ there exists $e \in [k-1]$ and $i \in [2^e]$ such that $|S_{e,i}| \geq 2$.
- $\text{BadE}_2 \Leftrightarrow$ there exists $i \in [2^k]$ such that $|S_{k,i}| \geq 2\lambda / \log \lambda$.

Since the probability of querying a fixed commitment is $1/(2^\lambda - P)$, by Lemma 5 and (11),

$$\Pr[\text{BadE}_1 \vee \text{BadE}_2] \leq \frac{14P}{2^\lambda} \quad (14)$$

If the bad events do not happen, $\mathcal{A}$ wins the game only if $|S| > u$ as S contains all possible (pdecom_I, I) . Then by $\neg(\text{BadE}_1 \vee \text{BadE}_2)$, we have

$$|S| \leq \sum_{i \in [2^{k-1}]} |S_{k,2i}| \cdot |A_{2i-1}| \leq \sum_{i \in [2^{k-1}]} |S_{k,2i}| \cdot |S_{k,2i-1}| \leq 2^{k+1} \left(\frac{\lambda}{\log \lambda} \right)^2.$$

Therefore, $\mathcal{A}$ cannot win the game without the bad events so we have

$$\text{Adv}_{\text{IC-VSC}}^{u\text{-esb}}(\mathcal{A}) \leq \Pr[\text{BadE}_1 \vee \text{BadE}_2] \leq \frac{14Q}{2^\lambda}$$

provided that $u = 2^{k+1} \left(\frac{\lambda}{\log \lambda} \right)^2$. $\square$

Now we provide the proof of the multi-instance hiding lemma. Since a collision between IVs matters in the Q_I -MIH game, the proof will be slightly different from [18]. As the proof is similar to that of [19], we give a sketch of the proof.

Lemma 2 (Multi-instance Hiding, [19]) *Let $E : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be an ideal cipher. Let $\mathcal{A}$ be an arbitrary adversary that makes P queries to E . Let 2^k as the vector size. Provided that $Q_I \leq 2^{80}$, $\hat{\ell}_{\text{blk}} \leq 2^{16}$, and $k \leq 16$, $\mathcal{A}$'s multi-instance hiding advantage $\text{Adv}_{\text{IC-VSC}}^{Q_I\text{-mih}}(\mathcal{A})$ against IC-VSC is bounded by*

$$\text{Adv}_{\text{IC-VSC}}^{Q_I\text{-mih}}(\mathcal{A}) \leq \frac{(3 + 5 \cdot 2^{2k} + (\hat{\ell}_{\text{blk}} + 7)^2 + k)Q_I}{2^\lambda} + \frac{(25 + 24k)P}{2^\lambda} + \frac{2}{2^\lambda}.$$

Proof (Sketch). We will bound the advantage using the H-coefficient technique. Denote $\mathcal{I}$ the ideal world where $b^* = 1$, and denote $\mathcal{R}$ the real world where $b^* = 0$. Let $\mathcal{T}$ be the transcript of $\mathcal{A}$ which contains queries to the ideal cipher and the instances given in the game. At the end of the game, root_{b^*} is given to the adversary for the convenience of probability computation. After root_{b^*} is given, an adversary learns ideal cipher key, input, output relations from the tree construction, commitment, and tape generation. If these queries are inconsistent, the adversary can distinguish the ideal from the real. Otherwise, the two worlds remain indistinguishable.

Define $\mathcal{L}$ as the set of the ideal cipher query made by the adversary. Let $\mathcal{L}_{\text{node}}$ (resp. $\mathcal{L}_{\text{com}}, \mathcal{L}_{\text{share}}$) denote the set of the ideal cipher key-input-output relations learned from the tree construction (resp. commitment, tape generation). Additionally, define $\mathcal{L}_{\text{ctr}}$ as $\mathcal{L}_{\text{com}} \cup \mathcal{L}_{\text{share}}$. If it is not confusing, we will omit subscript $j \in [Q_I]$ which is the instance index.

For each instance and $e \in \{2, \dots, k\}$, ideal cipher queries from node computation are added to $\mathcal{L}_{\text{node}}$. For each instance, ideal cipher queries from leaf commitment computation and hidden VOLE share computation are added to $\mathcal{L}_{\text{com}}$ and $\mathcal{L}_{\text{share}}$, respectively. If ideal cipher queries from the given instances have partial blocks, the corresponding full blocks will be given to $\mathcal{A}$. For $z \in \{0, 1\}^\lambda$, we define some subsets of $\mathcal{L}_{\text{node}}, \mathcal{L}_{\text{com}}, \mathcal{L}_{\text{share}}$ as follows.

$$\begin{aligned}\mathcal{L}_{\text{node}}[z] &= \{(K, X, Y) \in \mathcal{L}_{\text{node}} : K = z\}, \\ \mathcal{L}_{\text{ctr}}^+[z] &= \{(K, X, Y) \in \mathcal{L}_{\text{ctr}} : X = z\}, \\ \mathcal{L}_{\text{com}}^-[z] &= \{(K, X, Y) \in \mathcal{L}_{\text{com}} : Y = z\}.\end{aligned}$$

For each instance, parse $\text{iv} = (\text{iv}_{\text{pt}}, \text{iv}_{\text{key}}, \text{b})$ and define anc_e be the ancestor of $\text{node}_{k,i}$ (which is not replaced) in the GGM tree at depth $e \in [k]$. In the real world, anc_e is also an ancestor of $m_i[:\lambda]$ (which is replaced in the ideal world), or it is not otherwise. Let $\Delta = m_i[:\lambda] \oplus \text{node}_{k,i}$. By the correlation property, Δ remains the same across all instance and $\text{anc}_e = \text{node}_{e,i_e} \oplus \Delta$ for all $e \in [d]$, which is randomly sampled.

Now, we will define two kind of bad events; one is the restriction of (multi-)collisions (**Aux**), and the other is the real events (**Bad**). When one of the latter event happens, the adversary surely distinguishes both worlds. On the other hand, the former events are not critical to distinguishing itself but made for the proof easy. We define the former events as follows. We note that the probability bound of following events is computed assuming the ideal world since the probability of bad events in the ideal world should be bounded to use the H-coefficient technique.

- **Aux₁**: The maximal multi-collision in $\text{iv}_{\text{pt}}[:\lambda - 32]$ is greater than 12 or the maximal multi-collision in iv_{key} is greater than 12. From (12), the probability is bounded by

$$\Pr[\text{Aux}_1] \leq 2Q_I / (2^{16} \cdot 2^{2(\lambda-32)}) \leq Q_I / 2^{2\lambda-49}.$$

- Aux_2 : There is a collision of at least two whole IVs ($\text{iv}_{\text{pt}} \parallel \text{iv}_{\text{key}}$). The probability of this event is bounded by

$$\Pr[\text{Aux}_2] \leq Q_I^2/2^{2\lambda}.$$

- Aux_3 : During computing nodes in $\text{IC-VSC.Commit}(\text{iv}_j, \text{root}_1)$, there exists an input collision on E . In other words, there exists $(\text{iv}_{\text{key}}, \text{node} \oplus \text{iv}_{\text{pt}})$ such that $E_{\text{iv}_{\text{key}}}(\text{node} \oplus \text{iv}_{\text{pt}})$ is queried at least twice.

As all the root nodes are same, the probability that $(\text{iv}_{\text{key}}, \text{root}_1 \oplus \text{iv}_{\text{pt}})$ collides with $\neg\text{Aux}_2$ is 0. Two distinct node evaluations $E_{\text{iv}_{\text{key}}}(\text{node} \oplus \text{iv}_{\text{pt}})$ and $E_{\text{iv}'_{\text{key}}}(\text{node}' \oplus \text{iv}'_{\text{pt}})$ from different instances incur an input collision at their child node levels if either of their child nodes collide and keys collide. In either cases, child node pair – left or right – will collide with probability at most

$$\frac{2}{2^\lambda(2^\lambda - 24 \cdot 2^k)}$$

with $\neg\text{Aux}_1$. Similarly, two nodes in a single instance collides with probability at most

$$\frac{2}{2^\lambda - 24 \cdot 2^k}.$$

By a hybrid argument, we have

$$\begin{aligned} \Pr[\text{Aux}_3 \wedge \neg\text{Aux}_1 \wedge \neg\text{Aux}_2] &\leq \binom{Q_I}{2} \binom{2^{k+1}}{2} \cdot \frac{2}{2^\lambda(2^\lambda - 24 \cdot 2^k)} + \binom{2^{k+1}}{2} \cdot \frac{2Q_I}{2^\lambda - 24 \cdot 2^k} \\ &\leq \frac{4 \cdot 2^{2k} Q_I^2}{2^{2\lambda}} + \frac{4 \cdot 2^{2k} Q_I}{2^\lambda} \leq \frac{5 \cdot 2^{2k} Q_I}{2^\lambda} \end{aligned}$$

provided that $k \leq \lambda - 1$.

- Aux_4 : The maximal multi-collision in $\text{com}_{\bar{i}}$ is greater than 12. As there are at most

$$((\hat{\ell}_{\text{blk}} + 1)2^k + (2^k - 1))Q_I \leq 2^{\lambda-1}.$$

queries to E made during Commit , on top of $\neg\text{Aux}_3$, the probability that $\text{com}_{\bar{i}} = x$ for some $x \in \{0, 1\}^\lambda$ is at most $2/2^\lambda$. From (12), we have

$$\Pr[\text{Aux}_4 \wedge \neg\text{Aux}_3] \leq Q_I/(2^{16} \cdot 2^{2\lambda-1}) \leq Q_I/2^{2\lambda+15}.$$

Now we introduce the bad events. As a single incompatible ideal cipher query distinguishes the two worlds, each bad event corresponds to two of $\{\mathcal{L}, \mathcal{L}_{\text{ctr}}, \mathcal{L}_{\text{node}}\}$ queries collide. As a collision between ideal cipher queries made by the adversary – both in $\mathcal{L}$ – does not contribute to the distinguishing advantage, there are 5 bad events.

- **Bad₁**: There exists $(K, X, Y) \in \mathcal{L}$ and $(K', X', Y') \in \mathcal{L}_{\text{ctr}}$ such that $(K, X) = (K', X')$ or $(K, Y) = (K', Y')$. Since K' is randomly chosen and Y' is randomly chosen if it is from $\mathcal{L}_{\text{share}}$, we have

$$\Pr[(K, X) = (K', X')] \leq \begin{cases} \frac{1}{2^\lambda} & \text{if } (K', X', Y') \in \mathcal{L}_{\text{ctr}}^+[X], \\ 0 & \text{otherwise} \end{cases}$$

$$\Pr[(K, Y) = (K', Y')] \leq \begin{cases} \frac{1}{2^\lambda} & \text{if } (K', X', Y') \in \mathcal{L}_{\text{com}}^-[Y], \\ \frac{1}{2^{2\lambda}} & \text{if } (K', X', Y') \in \mathcal{L}_{\text{share}}, \\ 0 & \text{otherwise.} \end{cases}$$

By assuming $\neg(\text{Aux}_1 \vee \text{Aux}_4)$, we get

$$\Pr[\text{Bad}_1 \wedge \neg(\text{Aux}_1 \vee \text{Aux}_4)] \leq \frac{24P}{2^\lambda} + \frac{\hat{\ell}_{\text{blk}} Q_I P}{2^{2\lambda}} \leq \frac{25P}{2^\lambda}.$$

- **Bad₂**: There exists $(K, X, Y) \in \mathcal{L}$ and $(K, X', Y') \in \mathcal{L}_{\text{node}}$ such that $X = X'$ or $Y = Y'$. For each query $(K, X, Y) \in \mathcal{L}$ and $(K, X', Y') \in \mathcal{L}_{\text{node}}[K]$, we have

$$\Pr[X = X' \vee Y = Y'] \leq \frac{2}{2^\lambda}.$$

by the randomness of Δ . By assuming $\neg \text{Aux}_1$, we get $|\mathcal{L}_{\text{node}}[K]| \leq 12k$ and

$$\Pr[\text{Bad}_2 \wedge \neg \text{Aux}_1] \leq \sum_{K \in \{0,1\}^\lambda} \leq \frac{24k \cdot P}{2^\lambda}.$$

- **Bad₃**: There exist distinct triples $(K, X, Y), (K', X', Y') \in \mathcal{L}_{\text{ctr}}$ such that $(K, X) = (K', X')$ or $(K, Y) = (K', Y')$. If (K, X, Y) and (K', X', Y') are from the same instance, we trivially have $K = K'$, $X \neq X'$, and $\Pr[Y = Y'] = 1/2^\lambda$, so

$$\Pr[\text{Bad}_3 \text{ occurs in a same instance}] \leq \binom{\hat{\ell}_{\text{blk}} + 1}{2} \frac{Q_I}{2^\lambda}.$$

Now, assume that (K, X, Y) and (K', X', Y') are from different instance. **Bad₃** occurs in different instances if two queries both in $\mathcal{L}_{\text{ctr}}^+[X]$ or $\mathcal{L}_{\text{com}}^-[Y]$ collide at key, or a query in $\mathcal{L}_{\text{ctr}}$ and a query in $\mathcal{L}_{\text{share}}$ collide at key and output. Since $\neg \text{Aux}_3$ prevent the input collision while computing nodes in IC-VSC.Commit, we have $\Pr[K = K' \wedge \neg \text{Aux}_3] \leq \frac{2}{2^\lambda}$ and the probability is bounded by

$$\begin{aligned} & \Pr[\text{Bad}_3 \text{ occurs in different instances} \wedge \neg(\text{Aux}_1 \vee \text{Aux}_3 \vee \text{Aux}_4)] \\ & \leq \frac{12(\hat{\ell}_{\text{blk}} + 2)Q_I}{2^\lambda} + \frac{(\hat{\ell}_{\text{blk}} + 1)^2 Q_I^2}{2^{2\lambda}}. \end{aligned}$$

All in all, we have

$$\Pr[\text{Bad}_3 \wedge \neg(\text{Aux}_1 \vee \text{Aux}_3 \vee \text{Aux}_4)] \leq \frac{(\hat{\ell}_{\text{blk}} + 7)^2 Q_I}{2^\lambda}.$$

- **Bad₄**: There exist distinct triples $(K, X, Y) \in \mathcal{L}_{\text{node}}$ and $(K', X', Y') \in \mathcal{L}_{\text{ctr}}$ such that $(K, X) = (K', X')$ or $(K, Y) = (K', Y')$. Since

$$\Pr \left[\Delta \xleftarrow{\$} \{0, 1\}^\lambda : X = X' \vee Y = Y' \right] \leq \frac{2}{2^\lambda}$$

$$\Pr \left[\text{iv}_{\text{key}} \xleftarrow{\$} \{0, 1\}^\lambda : K = K' \mid \Delta = \delta \right] \leq \frac{1}{2^\lambda}$$

for any fixed δ , we have

$$\Pr [\text{Bad}_4] \leq \frac{2|\mathcal{L}_{\text{node}}| \cdot |\mathcal{L}_{\text{ctr}}|}{2^{2\lambda}} \leq \frac{2(\hat{\ell}_{\text{blk}} + 1)k \cdot Q_I^2}{2^{2\lambda}} \leq \frac{2Q_I}{2^\lambda}$$

- **Bad₅**: There exist distinct triples $(K, X, Y), (K', X', Y') \in \mathcal{L}_{\text{node}}$ such that $(K, X) = (K', X')$ or $(K, Y) = (K', Y')$. Since $\text{anc}_{e-1} = \text{node}_{e-1, i_{e-1}} \oplus \Delta$ implies

$$(\text{iv}_{\text{key}}, \text{anc}_{e-1} \oplus \text{iv}_{\text{pt}}) = (\text{iv}'_{\text{key}}, \text{anc}'_{e'-1} \oplus \text{iv}'_{\text{pt}})$$

$$\Leftrightarrow (\text{iv}_{\text{key}}, \text{node}_{e-1, i_{e-1}} \oplus \text{iv}_{\text{pt}}) = (\text{iv}'_{\text{key}}, \text{node}_{e'-1, i'_{e'-1}} \oplus \text{iv}'_{\text{pt}}),$$

the probability of the former event is computed by

$$\Pr [(K, X) = (K', X') \wedge \neg \text{Aux}_3] = 0.$$

Similarly, by using the fact that $\text{anc}_{e-1} = \text{node}_{e-1, i_{e-1}} \oplus \Delta$, we have

$$\Pr [(K, Y) = (K', Y') \wedge \neg \text{Aux}_3] \leq \frac{1}{2^{2\lambda}}.$$

All in all, we get

$$\Pr [\text{Bad}_5 \wedge \neg \text{Aux}_3] \leq \frac{(k \cdot Q_I)^2}{2^{2\lambda}} \leq \frac{k \cdot Q_I}{2^\lambda}$$

With $\text{Aux} = \text{Aux}_1 \vee \dots \vee \text{Aux}_4$ and $\text{Bad} = \text{Bad}_1 \vee \dots \vee \text{Bad}_5$, we have

$$\begin{aligned} \Pr [\text{Aux} \vee \text{Bad}] &\leq \frac{Q_I}{2^{2\lambda-33}} + \frac{Q_I^2}{2^{2\lambda}} + \frac{5 \cdot 2^{2k} Q_I}{2^\lambda} + \frac{Q_I}{2^{2\lambda+15}} \\ &\quad + \frac{25Q}{2^\lambda} + \frac{24k \cdot Q}{2^\lambda} + \frac{(\hat{\ell}_{\text{blk}} + 7)^2 Q_I}{2^\lambda} + \frac{2Q_I}{2^\lambda} + \frac{k \cdot Q_I}{2^\lambda} \\ &\leq \frac{(3 + 5 \cdot 2^{2k} + (\hat{\ell}_{\text{blk}} + 7)^2 + k)Q_I}{2^\lambda} + \frac{(25 + 24k)P}{2^\lambda} + \frac{2}{2^\lambda} \end{aligned}$$

where the inequalities assume that $\lambda \geq 78$.

We say $\mathcal{T}_{\text{Bad}}$ be the set of bad transcripts, while $\mathcal{T}_{\text{Good}}$ be the complement of $\mathcal{T}_{\text{Bad}}$, and let T_{id} (resp. T_{re}) be the distribution of γ in $\mathcal{I}$ (resp. $\mathcal{R}$). As P ideal cipher queries and Q_I instances of $m_{\hat{i}}$ are included in transcripts, for $\gamma \in \mathcal{T}_{\text{Good}}$,

$$\Pr [\gamma = T_{\text{id}}] = \frac{1}{(2^{(\hat{\ell}_{\text{blk}}+1)\lambda})^{Q_I}} \cdot \prod_{s \in \{0,1\}^\lambda} \frac{1}{(2^\lambda)^{P_s}},$$

where P_s denotes the number of ideal cipher queries with key input s . Additionally, depending on oracle queries, some values may be excluded as candidates for $m_{\bar{i}}$,

$$\Pr[\gamma = T_{\text{re}}] \geq \frac{1}{(2^{(\hat{\ell}_{\text{blk}}+1)\lambda})^{Q_I}} \cdot \prod_{s \in \{0,1\}^\lambda} \frac{1}{(2^\lambda)^{P_s}},$$

where P_s denotes the number of ideal cipher queries with key input s . Therefore, by Lemma 4, the advantage is bounded by

$$\text{Adv}_{\text{IC-VSC}}^{Q_I\text{-mih}}(\mathcal{A}) \leq \frac{(3 + 5 \cdot 2^{2k} + (\hat{\ell}_{\text{blk}} + 7)^2 + k)Q_I}{2^\lambda} + \frac{(25 + 24k)P}{2^\lambda} + \frac{2}{2^\lambda}.$$

□

B.2 Proof of Claims in Lemma 3

In this section, we provide the proofs of two claims in Lemma 3. The proofs follow the assumptions and notations used in Lemma 3.

Claim 1 Fix $\text{chall}_1 \in \{0,1\}^{5\lambda+64}$, $\text{iv} \in \{0,1\}^{\lambda-32}$, $\overline{\text{com}} \in \{0,1\}^\lambda$, $C \in \{0,1\}^{\lambda+B}$. Then, for a fresh PRG query $\text{PRG}_{i,j}(\text{seed}, \text{iv})$, one have

$$\Pr \left[\begin{array}{l} E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, 0]) = \overline{\text{com}} \\ \text{VOLEHash}(\text{chall}_1, \text{PRG}_{i,j}(\text{seed}, \text{iv})) = C \end{array} \right] \leq \frac{8}{2^{2\lambda+B}}.$$

Proof. For a fixed chall_1 , we can rephrase $\text{VOLEHash}(\text{chall}_1, x_0 \parallel x_1)$ where $x_0 \in \{0,1\}^*$ and $x_1 \in \{0,1\}^{\lambda+B}$ as $F(x_0) \oplus x_1$ for some $F : \{0,1\}^* \rightarrow \{0,1\}^{\lambda+B}$. We divide the cases based on how the PRG query was initiated.

CASE 1. If $\mathcal{A}$ makes forward query $E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, \cdot])$, we first evaluate $E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, x])$ for $x = 1, \dots, \ell_{\text{blk}} - 2$ (recall that we gave the full PRG query instead of single ideal cipher query for this case). Then, $\text{PRG}_{i,j}(\text{seed}, \text{iv})[1 : \ell_{\text{blk}} - 2]$ is fixed, and we should upper bound the probability to have

$$\begin{cases} E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, 0]) = \overline{\text{com}} \\ E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, \ell_{\text{blk}} - 1]) \parallel E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, \ell_{\text{blk}}])[: B] = C \oplus F(x_0) \end{cases}$$

where $x_0 = \text{seed} \parallel \text{PRG}_{i,j}(\text{seed}, \text{iv})[1 : \ell_{\text{blk}} - 2]$, which is at most

$$\frac{2^{\lambda-B}}{(2^\lambda - (\ell_{\text{blk}} + 1)P)^3} \leq \frac{8}{2^{2\lambda+B}}$$

provided that $(\ell_{\text{blk}} + 1)P \leq 2^{\lambda-1}$.

CASE 2. If $\mathcal{A}$ makes backward query $E_{\text{seed}}^{-1}(\overline{\text{com}})$, we first require

$$E_{\text{seed}}^{-1}(\overline{\text{com}}) = \text{iv} \parallel \text{ctr}[i, j, 0]$$

which happens with probability at most

$$\frac{1}{2^\lambda - (\ell_{\text{blk}} + 1)P}.$$

The remaining part is similar to Case 1. If we evaluate $E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, x])$ for $x = 1, \dots, \ell_{\text{blk}} - 2$, we should upper bound the probability to have

$$E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, \ell_{\text{blk}} - 1]) \parallel E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, \ell_{\text{blk}}])[B] = C \oplus F(x_0)$$

where $x_0 = \text{seed} \parallel \text{PRG}_{i,j}(\text{seed}, \text{iv})[1 : \ell_{\text{blk}} - 2]$. Then, the probability is upper bounded by

$$\frac{2^{\lambda-B}}{(2^\lambda - (\ell_{\text{blk}} + 1)P)^3} \leq \frac{8}{2^{2\lambda+B}}$$

provided that $(\ell_{\text{blk}} + 1)P \leq 2^{\lambda-1}$.

CASE 3. If $\mathcal{A}$ makes backward query $E_{\text{seed}}^{-1}(Y)$ where $Y \neq \overline{\text{com}}$ we first require

$$E_{\text{seed}}^{-1}(Y) = \text{iv} \parallel \text{ctr}[i, j, a]$$

for $a \in \{1, \dots, \ell_{\text{blk}}\}$. If $a \leq \ell_{\text{blk}} - 2$, it become same as Case 1. Now assume that $a = \ell_{\text{blk}} - 1$, which happens with probability at most

$$\frac{1}{2^\lambda - (\ell_{\text{blk}} + 1)P}$$

Then, after evaluating $\text{PRG}_{i,j}(\text{seed}, \text{iv})[1 : \ell_{\text{blk}} - 1]$, we should upper bound the probability to have

$$\begin{cases} E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, 0]) = \overline{\text{com}} \\ E_{\text{seed}}(\text{iv} \parallel \text{ctr}[i, j, \ell_{\text{blk}}])[B] = (C \oplus F(x_0))[\lambda : \lambda + B] \end{cases}$$

where $x_0 = \text{seed} \parallel \text{PRG}_{i,j}(\text{seed}, \text{iv})[1 : \ell_{\text{blk}} - 2]$. Then, the probability is upper bounded by

$$\frac{2^{\lambda-B}}{(2^\lambda - (\ell_{\text{blk}} + 1)P)^3} \leq \frac{8}{2^{2\lambda+B}}$$

provided that $(\ell_{\text{blk}} + 1)P \leq 2^{\lambda-1}$. If $a = \ell_{\text{blk}}$, all are similar but we get the upper bound of probability as following:

$$\frac{1}{(2^\lambda - (\ell_{\text{blk}} + 1)P)^3} \leq \frac{8}{2^{3\lambda}}$$

provided that $(\ell_{\text{blk}} + 1)P \leq 2^{\lambda-1}$. All in all, the claim holds for any ideal cipher queries from $\mathcal{A}$. $\square$

Now we provide the proof of Claim 2. It claims that the ChosenVOLECommit extractor cannot extract two consistent VOLE pairs within a single repetition unless G_3 aborts.

Claim 2 *Ext does not return $\perp_{\text{abort}}$ in Step 6 unless G_3 aborts.*

Proof. Fix $i \in [\tau]$ and distinct $(\mathbf{u}, \mathbf{V}), (\mathbf{u}', \mathbf{V}') \in \mathcal{E}_i$. Let S_Δ and $S_{\Delta'}$ be the set of possible opening indices of $(\mathbf{u}, \mathbf{V})$ and $(\mathbf{u}', \mathbf{V}')$, respectively. Assume that there exist $\Delta = (\delta_0, \dots, \delta_{k-1}) \in S_\Delta$ and $\Delta' = (\delta'_0, \dots, \delta'_{k-1}) \in S_{\Delta'}$ such that

$$\begin{cases} (\text{VOLEHash}(\text{chall}_1, \mathbf{u} \oplus \mathbf{c}_i) \oplus \tilde{\mathbf{u}}) \delta_j = \text{VOLEHash}(\text{chall}_1, \mathbf{V}[j]) \oplus \tilde{\mathbf{V}}[j] \\ (\text{VOLEHash}(\text{chall}_1, \mathbf{u}' \oplus \mathbf{c}_i) \oplus \tilde{\mathbf{u}}) \delta'_j = \text{VOLEHash}(\text{chall}_1, \mathbf{V}'[j]) \oplus \tilde{\mathbf{V}}[j] \end{cases} \quad (15)$$

holds for all $j \in [k]$. Let $(\mathbf{r}_0, \dots, \mathbf{r}_{N-1})$ and $(\mathbf{r}'_0, \dots, \mathbf{r}'_{N-1})$ where $\mathbf{r}_\Delta = \mathbf{r}'_{\Delta'} = 0$ be punctured VOLE shares of $(\mathbf{u}, \mathbf{V})$ and $(\mathbf{u}', \mathbf{V}')$, respectively. Then, by Property 1 (with $\neg \text{BadE}_1 \wedge \neg \text{BadE}_2$), we have $\mathbf{r}_j = \mathbf{r}'_j$ for $j \in [N] \setminus \{\Delta, \Delta \oplus 1, \Delta', \Delta' \oplus 1\}$.

CASE 1: $\delta_0 = \delta'_0 = 0$. It implies that both Δ and Δ' are even numbers. We have $\text{VOLEHash}(\text{chall}_1, \mathbf{V}[0] \oplus \mathbf{V}'[0]) = 0$ which implies $\mathbf{V}[0] = \mathbf{V}'[0]$ (by the abort condition in G_3) and

$$\begin{cases} \mathbf{r}_{\Delta \oplus 1} \oplus \mathbf{r}'_{\Delta \oplus 1} \oplus \mathbf{r}_{\Delta' \oplus 1} \oplus \mathbf{r}'_{\Delta' \oplus 1} = 0 & \text{if } \Delta \neq \Delta' \\ \mathbf{r}_{\Delta \oplus 1} \oplus \mathbf{r}'_{\Delta \oplus 1} = 0 & \text{if } \Delta = \Delta' \end{cases}$$

By the modification in game G_2 , it holds only if the seeds are pairwise same in the same position, i.e., $(\mathbf{r}_{\Delta \oplus 1}, \mathbf{r}_{\Delta' \oplus 1}) = (\mathbf{r}'_{\Delta \oplus 1}, \mathbf{r}'_{\Delta' \oplus 1})$.

CASE 2: $\delta_0 = \delta'_0 = 1$. It implies that both Δ and Δ' are odd numbers. We have $\text{VOLEHash}(\text{chall}_1, \mathbf{u} \oplus \mathbf{V}[0] \oplus \mathbf{u}' \oplus \mathbf{V}'[0]) = 0$ which implies $\mathbf{u} \oplus \mathbf{V}[0] = \mathbf{u}' \oplus \mathbf{V}'[0]$, and we have

$$\begin{cases} \mathbf{r}_{\Delta \oplus 1} \oplus \mathbf{r}'_{\Delta \oplus 1} \oplus \mathbf{r}_{\Delta' \oplus 1} \oplus \mathbf{r}'_{\Delta' \oplus 1} = 0 & \text{if } \Delta \neq \Delta' \\ \mathbf{r}_{\Delta \oplus 1} \oplus \mathbf{r}'_{\Delta \oplus 1} = 0 & \text{if } \Delta = \Delta'. \end{cases}$$

Similarly as Case 1, we have $(\mathbf{r}_{\Delta \oplus 1}, \mathbf{r}_{\Delta' \oplus 1}) = (\mathbf{r}'_{\Delta \oplus 1}, \mathbf{r}'_{\Delta' \oplus 1})$.

CASE 3: $\delta_0 \oplus \delta'_0 = \delta_j \oplus \delta'_j = 1$. We have $\text{VOLEHash}(\text{chall}_1, \mathbf{V}[0] \oplus \mathbf{V}[j] \oplus \mathbf{V}'[0] \oplus \mathbf{V}'[j]) = 0$ which implies $\mathbf{V}[0] \oplus \mathbf{V}[j] \oplus \mathbf{V}'[0] \oplus \mathbf{V}'[j] = 0$. Then, either one of following condition holds:

- If $\delta_0 \oplus \delta_j = 0$, then $\mathbf{r}_{\Delta \oplus 1} \oplus \mathbf{r}'_{\Delta \oplus 1} \oplus \mathbf{r}_{\Delta' \oplus 1} \oplus \mathbf{r}'_{\Delta' \oplus 1} = 0$, which implies $(\mathbf{r}_{\Delta \oplus 1}, \mathbf{r}_{\Delta' \oplus 1}) = (\mathbf{r}'_{\Delta \oplus 1}, \mathbf{r}'_{\Delta' \oplus 1})$.
- Otherwise, $\mathbf{r}_\Delta \oplus \mathbf{r}'_\Delta \oplus \mathbf{r}_{\Delta'} \oplus \mathbf{r}'_{\Delta'} = 0$, which implies $\mathbf{r}_{\Delta'} = \mathbf{r}'_{\Delta'}$ (contradiction by the modification in G_2).

Putting them together, the above cases show that $\mathbf{r}_j = \mathbf{r}'_j$ for all $j \in [N] \setminus \{\Delta, \Delta'\}$ ⁶, which implies

$$|\mathcal{S}_i^{(2)}| \leq 1.$$

Now, suppose $|\mathcal{S}_i^{(2)}| = 0$, and we divide two cases.

⁶ We note that the untreated case $-\Delta \oplus \Delta' = 1$ is also in this case.

- Suppose there exists such $(\mathbf{u}, \mathbf{V}), (\mathbf{u}', \mathbf{V}')$ with distinct Δ, Δ' that satisfies (15). Let $(\mathbf{u}'', \mathbf{V}'') = \text{ConvertToVOLE}(\mathbf{r}_0'', \dots, \mathbf{r}_{k-1}'')$ where $\mathbf{r}_j'' = \mathbf{r}_j$ for $j \neq \Delta$, and $\mathbf{r}_\Delta'' = \mathbf{r}'_\Delta$. Then,

$$(\text{VOLEHash}(\text{chall}_1, \mathbf{u}'' \oplus \mathbf{c}_i) \oplus \tilde{\mathbf{u}})\Delta'' = \text{VOLEHash}(\text{chall}_1, \mathbf{V}'') \oplus \tilde{\mathbf{V}}$$

holds for both $\Delta'' = \Delta$ and Δ' , so

$$\text{VOLEHash}(\text{chall}_1, \mathbf{u}'' \oplus \mathbf{c}_i) = \tilde{\mathbf{u}}.$$

As it violates the assumption that $|\mathcal{S}_i^{(2)}| \leq 1$, this case does not happen.

- Suppose that there exists unique opening index Δ that passes consistency check for all $(\mathbf{u}, \mathbf{V}) \in \mathcal{E}_i$. Then, we simply have $|\mathcal{S}_i^{(1)}| = 1$.

All in all, we have either $|\mathcal{S}_i^{(2)}| = 1$ or $(|\mathcal{S}_i^{(1)}|, |\mathcal{S}_i^{(2)}|) = (1, 0)$, so Ext does not return $\perp_{\text{abort}}$ in Step 6. $\square$

C Full Proof of the EUF-CMA Security

We provide the EUF-KO security proof of rVOLEitH. As we proved the extractable binding property of ChosenVOLECommit in Lemma 3, what we have left is ensuring that there is no unfortunate event on ZKHash. The rest part of the proof is semantically same with the corresponding part in [4].

Theorem 1 (EUF-KO Security of rVOLEitH). *Let param be parameters of rVOLEitH. Assume that H_1, H_2^j for $j \in \{1, 2, 3\}$, H_3 , and H_4 are modeled as random oracles. Let $\mathcal{A}$ be an arbitrary adversary against the EUF-KO security of rVOLEitH that makes a total of Q random oracle queries and P ideal cipher queries. Let $Q_1, Q_{2,j}, Q_3$, and Q_4 be the number of queries to H_1, H_2^j, H_3 , and H_4 made by $\mathcal{A}$. Then there exists PPT adversary $\mathcal{B}$ against OWF security of KeyGen such that*

$$\begin{aligned} \text{Adv}_{\text{rVOLEitH}}^{\text{euf-ko}}(\mathcal{A}) &\leq Q_{2,2} \left(\varepsilon_{\text{zk}} + \frac{8uP}{2^{2\lambda+B}} \right) + \frac{2Q_{2,3}}{2^\lambda} + \text{Adv}_{\text{KeyGen}}^{\text{owf}}(\mathcal{B}) \\ &\quad + \frac{Q^2}{2^{2\lambda}} + \frac{18P}{2^\lambda} + \frac{\tau u^2 \varepsilon_v (k + \tau + 1) Q_{2,1}}{2} + \frac{8u^2 (k + \tau) Q_{2,1} P}{2^{2\lambda+B}} \end{aligned}$$

where $u = 2N(\lambda / \log \lambda)^2$, and $\ell_{\text{blk}} P \leq 2^{\lambda-1}$.

Proof. Let $\mathcal{A}$ be an EUF-KO adversary against rVOLEitH for given pk. Let $\mathcal{L}_E, \mathcal{L}_1, (\mathcal{L}_2^i)_{i=1,2,3}, \mathcal{L}_3$, and $\mathcal{L}_4$ be the record of query-response pairs made by $\mathcal{A}$ to $E, H_1, (H_2^i)_{i=1,2,3}, H_3$ and H_4 , respectively. We argue by game hops. Let $\mathcal{G}_0$ be the original EUF-KO game.

$\mathcal{G}_1$: The game keep tracks all ideal cipher queries and random oracle queries made by $\mathcal{A}$, and aborts if $\mathcal{L} = (\mathcal{L}_E, \mathcal{L}_1, \mathcal{L}_2^1, \mathcal{L}_4) \in \Gamma_{\text{Bad}}$ (defined in Lemma 3). By Lemma 3, we have

$$\Pr[\mathcal{G}_1 \text{ aborts}] \leq \frac{Q^2}{2^{2\lambda}} + \frac{18P}{2^\lambda} + \frac{\tau u^2 \varepsilon_v (k + \tau + 1) Q_{2,1}}{2} + \frac{8u^2 (k + \tau) Q_{2,1} P}{2^{2\lambda+B}}.$$

G_2 : When $(\text{chall}_1, \tilde{\mathbf{u}}, h_V, \mathbf{d})$ is queried to H_2^2 , the game runs the extractor Ext of ChosenVOLECommit in Lemma 3 to extract the VOLEs $(\mathbf{u}^*, \mathbf{V}^*)$ and remembers the result. If the adversary's forgery is valid, but was not extracted before, the game aborts. By Lemma 3 and the modification in G_1 , we have

$$\Pr[G_2 \text{ aborts}] \leq \frac{8uPQ_{2,2}}{2^{2\lambda+B}}.$$

Now $(\mathbf{u}^*, \mathbf{V}^*)$ is the unique VOLE pair which passes the consistency check because of the extractable binding property.

G_3 : When $(\text{chall}_1, \tilde{\mathbf{u}}, h_V, \mathbf{d})$ is queried to H_2^2 by $\mathcal{A}$ and extraction succeeds, the game samples $\text{chall}_2 \in \{0, 1\}^{3\lambda+64}$ and aborts if zero-knowledge soundness check was bad. More precisely, after setting $\mathbf{w} = \mathbf{u}^*[\lambda : \ell] \parallel (\mathbf{d} \oplus \mathbf{u}^*[\lambda : \ell])$, if $\mathbf{w}$ does not satisfy the given multiplication constraints (from OWF) but ZKHash corrects the error into a valid VOLE correlation, the game aborts. By the ε_{zk} -universality of ZKHash , we have

$$\Pr[G_3 \text{ aborts}] \leq \varepsilon_{\text{zk}} Q_{2,2}.$$

Note that this game is identical to Game G_6 in [4, Theorem 1].

G_4 : When $(\text{chall}_2, \tilde{a}, \tilde{b})$ is queried to H_2^3 by $\mathcal{A}$, if Quicksilver check failed to detect the invalid $\mathbf{w}$ (computed in G_3) because of Δ from chall_3 is a root of the quadratic polynomial in Quicksilver check, the game aborts. Since we consider only quadratic polynomials in Quicksilver check, we have

$$\Pr[G_4 \text{ aborts}] \leq \frac{2Q_{2,3}}{2^\lambda}.$$

Note that this game is identical to Game G_8 in [4, Theorem 1].

If $\mathcal{A}$ wins at G_4 , $\mathcal{A}$ succeeds to find the witness $\mathbf{w}$, which is consistent to given multiplication constraints. Therefore, there exists PPT adversary $\mathcal{B}$ against OWF security such that

$$\begin{aligned} \text{Adv}_{\text{rVOLEitH}}^{\text{euf-ko}}(\mathcal{A}) &\leq \sum_{i=1}^4 \Pr[G_i \text{ aborts}] + \text{Adv}_{\text{KeyGen}}^{\text{owf}}(\mathcal{B}) \\ &\leq Q_{2,2} \left(\varepsilon_{\text{zk}} + \frac{8uP}{2^{2\lambda+B}} \right) + \frac{2Q_{2,3}}{2^\lambda} + \text{Adv}_{\text{KeyGen}}^{\text{owf}}(\mathcal{B}) \\ &\quad + \frac{Q^2}{2^{2\lambda}} + \frac{18P}{2^\lambda} + \frac{\tau u^2 \varepsilon_v (k + \tau + 1) Q_{2,1}}{2} + \frac{8u^2 (k + \tau) Q_{2,1} P}{2^{2\lambda+B}} \end{aligned}$$

which concludes the proof. $\square$

Now we provide the EUF-CMA security proof of rVOLEitH . The most of the game in the proof is a kind of routine random string replacement. One different point of the routine process is that rVOLEitH uses 2λ bits iv so that the replacement is much easier than in [4]. G_6 is the only exception of such routine processes.

Theorem 2 (EUF-CMA Security of rVOLEitH). *Let param be parameters of rVOLEitH. Assume that $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$, H_2^1, H_2^2 , and H_2^3 are modeled as random oracles. For a PPT adversary $\mathcal{A}$ against the EUF-CMA security of rVOLEitH with a total of Q_{sig} signing oracle queries, Q random oracle queries, and P ideal cipher queries, there exist PPT adversaries*

- $\mathcal{B}$ against the EUF-KO security of rVOLEitH,⁷
- $\mathcal{C}$ against the PRF security of H_3 ,⁸
- $\mathcal{D}$ against the multi-instance hiding security of IC-VSC

such that

$$\begin{aligned} \text{Adv}_{\text{rVOLEitH}}^{\text{euf-cma}}(\mathcal{A}) &\leq \frac{Q_{\text{sig}}(Q_{\text{sig}} + Q)}{2^{2\lambda}} + Q_{\text{sig}} \cdot \text{Adv}_{H_3}^{\text{prf}}(\mathcal{C}) \\ &\quad + \text{Adv}_{\text{IC-VSC}}^{\tau Q_{\text{sig}}\text{-mih}}(\mathcal{D}) + \text{Adv}_{\text{rVOLEitH}}^{\text{euf-ko}}(\mathcal{B}). \end{aligned}$$

Proof. Let $\mathcal{A}$ be an EUF-CMA adversary against rVOLEitH for given pk . Let G_0 be the original EUF-CMA game. Let $\mathcal{O}_{\text{sig}}$ be the signing oracle, let $Q_{\text{sig}}, Q_1, Q_{2,1}, Q_{2,2}$, and $Q_{2,3}$ be the number of queries made to $\mathcal{O}_{\text{sig}}, H_1, H_2^1, H_2^2$, and H_2^3 by $\mathcal{A}$, respectively. We begin to prove the security of the deterministic version of rVOLEitH ($\rho \leftarrow 0^n$), and prove that of the probabilistic version later. Without loss of generality, we assume that all messages in signing queries are distinct.

G_1 : $\mathcal{O}_{\text{sig}}$ samples $\text{iv} \in \{0, 1\}^{2\lambda}$ at random instead of computing $H_3(\text{sk} \parallel \mu \parallel \rho)$ and $H_4(\text{iv})$. As μ is distinct for each query, by a hybrid argument, the difference between this game and the previous one reduces to the PRF security of H_3 with secret key sk . Therefore, there exists a PPT adversary $\mathcal{C}$ against the PRF security of H_3 such that

$$|\Pr[\mathcal{A} \text{ wins } G_1] - \Pr[\mathcal{A} \text{ wins } G_2]| \leq Q_{\text{sig}} \cdot \text{Adv}_{H_3}^{\text{prf}}(\mathcal{C})$$

G_2 : $\mathcal{O}_{\text{sig}}$ samples $\text{chall}_1 \in \{0, 1\}^{5\lambda+64}$ uniformly at random instead of computing

$$H_2^1(\mu \parallel h_{\text{com}} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}),$$

and also program the random oracles H_2^1 to output chall_1 for the respective query. The simulation is aborted if the queries to H_2^1 have been made previously. This game is indistinguishable with the previous game unless the simulation is aborted, as $\text{iv} \in \{0, 1\}^{2\lambda}$ is chosen uniformly at random the probability of abort is

$$\Pr[G_2 \text{ aborts}] \leq Q_{\text{sig}} \cdot \left(\frac{Q_{\text{sig}} + Q_{2,1}}{2^{2\lambda}} \right).$$

⁷ We assume $\mathcal{B}$ has the same amount of queries to random oracles.

⁸ H_3 itself is not a PRF, but it is used as a PRF with key prepending. We use this notation for convenience.

G_3 : $\mathcal{O}_{\text{sig}}$ samples $\text{chall}_2 \in \{0, 1\}^{3\lambda+64}$ uniformly at random instead of computing

$$H_2^2(\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel h_V \parallel \mathbf{d}),$$

and also program the random oracles H_2^2 to output chall_2 for the respective query. The simulation is aborted if the queries to H_2^2 have been made previously. This game is indistinguishable with the previous game unless the simulation is aborted, as $\text{chall}_1 \in \{0, 1\}^{5\lambda+64}$ is chosen uniformly at random the probability of abort is

$$\Pr[G_3 \text{ aborts}] \leq Q_{\text{sig}} \cdot \left(\frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda+64}} \right).$$

G_4 : $\mathcal{O}_{\text{sig}}$ samples $\text{chall}_3 \in \{0, 1\}^\lambda$ uniformly at random instead of computing

$$H_2^3(\text{chall}_2 \parallel \tilde{a} \parallel \tilde{b}),$$

and also program the random oracles H_2^3 to output chall_3 for the respective query. The simulation is aborted if the queries to H_2^3 have been made previously. This game is indistinguishable with the previous game unless the simulation is aborted, as $\text{chall}_2 \in \{0, 1\}^{3\lambda+64}$ are chosen uniformly at random the probability of abort is

$$\Pr[G_4 \text{ aborts}] \leq Q_{\text{sig}} \cdot \left(\frac{Q_{\text{sig}} + Q_{2,3}}{2^{3\lambda+64}} \right).$$

G_5 : $\mathcal{O}_{\text{sig}}$ samples $(\text{iv}_j)_{j \in [\tau]} \in (\{0, 1\}^{2\lambda})^\tau$ uniformly at random instead of computing

$$H_4(\text{iv})$$

and also program the random oracles H_4 to output $(\text{iv}_j)_{j \in [\tau]}$ for the respective query. The simulation is aborted if the queries to H_4 have been made previously. This game is indistinguishable with the previous game unless the simulation is aborted, as $\text{iv} \in \{0, 1\}^{2\lambda}$ is chosen uniformly at random the probability of abort is

$$\Pr[G_5 \text{ aborts}] \leq Q_{\text{sig}} \cdot \left(\frac{Q_{\text{sig}} + Q_4}{2^{2\lambda}} \right).$$

G_6 : $\mathcal{O}_{\text{sig}}$ randomly select $\text{root}^* \in \{0, 1\}^\lambda$ and replaces the call to $\text{VOLECommit}(\mathbf{w}[:\lambda], \text{iv}, \ell + 2\lambda + B)$ by call to $\text{VOLECommit}(\text{root}^*, \text{iv}, \ell + 2\lambda + B)$. And $\mathcal{O}_{\text{sig}}$ replaces hidden messages (seeds and corresponding VOLE shares) $\{m_{j, \bar{i}_j}\}_{j \in [\tau]}$ in VOLECommit by random strings, while preserving the correlations as follows.

$$m_{j, \bar{i}_j}[:\lambda] \leftarrow \mathbf{w}[:\lambda] - \sum_{i \in [2^k] \setminus \{\bar{i}_j\}} (m_{j, i}[:\lambda]).$$

The difference between G_5 and G_6 corresponds to the multi-instance hiding game of IC-VSC given τQ_{sig} instances, so there exists a PPT adversary $\mathcal{D}$ against the multi-instance hiding game such that

$$|\Pr[\mathcal{A} \text{ wins } G_5] - \Pr[\mathcal{A} \text{ wins } G_6]| \leq \mathbf{Adv}_{\text{IC-VSC}}^{\tau Q_{\text{sig}}\text{-mih}}(\mathcal{D})$$

We note that the distribution of copaths in pdecom_i becomes independent of secret key.

G_7 : $\mathcal{O}_{\text{sig}}$ now replaces $(\mathbf{u}[\lambda:], \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1})$ by a random element in $(\mathbb{F}_2^{\ell+\lambda+B})^\tau$. $\mathcal{O}_{\text{sig}}$ also replaces $\mathbf{V}$ by the element in $\mathbb{F}_2^{(\ell+2\lambda+B)\times\tau}$ such that the VOLE relation holds with the Δ and $\mathbf{q}_i$ values that result from chall_3 . Since the hidden messages $m_{j,\tilde{i}_j}$ are already replaced to random strings in the previous game, this game is perfectly indistinguishable from the previous game. After this game, the distribution of $\tilde{\mathbf{u}}$ and $\tilde{a}$ becomes uniformly random because of the hiding property of VOLEHash and ZKHash . The distribution of the commitment to the witness $\mathbf{d} = \mathbf{w}[\lambda:] \oplus \mathbf{u}[\lambda:]$ also becomes uniformly random since $\mathbf{u}[\lambda:]$ is chosen uniformly at random.

In G_7 , we see the the distribution of σ produced by $\mathcal{O}_{\text{sig}}$ no longer depends on the secret key sk and the success probability of $\mathcal{A}$ in this final game can be reduced to the EUF-KO security of rVOLEitH using $\mathcal{B}$.

For the non-deterministic version of $\mathcal{A}$, all games are defined in a manner almost identical to the deterministic version, with the exception of handling two queries to $\mathcal{O}_{\text{sig}}$ that involve the same messages and ρ values. If (m, ρ) are identical in two queries, the outputs must also be identical; thus, we avoid random sampling and use already programmed outputs for the random oracles in such cases. Consequently, the differences between the adjacent games remain unchanged from the deterministic version, leading to the same bounds on the advantage of $\mathcal{A}$.

□

D Detailed Attack Algorithm

We provide the detailed attack algorithm in the following.

1. Fix transcripts for the first challenge and some for the second challenge: $\text{msg}, (\delta_{y_j}, \delta_{z_j})_{j \in [\tau]}, A, B$, and $(a^{(j)})_{j \in [\tau]}$. b is not fixed yet. It fixes the hash outputs $h_0, \tilde{\mu}$, and $(\alpha_m)_{m \in [\tau D]}$.
2. Find $2^{\lambda/2}$ number of $(\vec{s}, \vec{w})$ pairs in $(\mathbb{F}_q)^{n-2}$ such that

$$\begin{aligned} \Phi'(\vec{s}, \vec{w}) &:= \left(\vec{u} \cdot \begin{pmatrix} \vec{s} \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{w} \\ 0 \\ 1 \end{pmatrix} \right) - \left(\vec{u} \cdot \begin{pmatrix} \vec{w} \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{s} \\ 0 \\ 1 \end{pmatrix} \right) \\ &\in \text{Span}(B\alpha_0, \dots, B\alpha_{\tau D-1}) + A \end{aligned}$$

where $\text{Span}(B\alpha_0, \dots, B\alpha_{\tau D-1}) + A$ means the $\mathbb{F}_2$ -linear span of $(B\alpha_0, \dots, B\alpha_{\tau D-1})$ shifted by A . Make a list L of triples $(\vec{s}, \vec{w}, \Phi'(\vec{s}, \vec{w}))$. One pair can be found by solving linear equation

$$\Phi'(\vec{s}_0, \vec{w}) = \sum_{i \in [\tau D]} c_i \alpha_i$$

with respect to $\vec{w}$ and c_i 's after fixing random $\vec{s} = \vec{s}_0$ with a significant probability.

3. Fix b . It fixes the hash outputs $h_1, (i_j^*)_{j \in [\tau]}, (\Delta_j^*)_{j \in [\tau]}$, and i_p^* . Check whether Δ^* satisfy $A + B\Delta^* = \Phi'(\vec{s}^*, \vec{w}^*)$ for some $(\vec{s}^*, \vec{w}^*, \Phi'(\vec{s}^*, \vec{w}^*)) \in L$. If not, do this step again with another b . This step will expectedly end in $2^{\lambda/2}$ repetitions.
4. Generate a pre-tree (with 2^τ leaves) using a random root, and fix it. This fixes one of two highest node in the each main tree (**node**_{1,2} in Figure 2), which is in the copath of i_j^* .
5. For $j \in [\tau]$, do the following:
 - (a) We need to find PPRF keys such that $A = A'$ and $(a^{(j)})_{j \in [\tau]} = (a^{(j)'})_{j \in [\tau]}$. Query to the random oracle (or ideal cipher), and record PPRF key candidates. For example, in Figure 2, query **node**_{2,1} and **seed**⁽⁴⁾, and record seeds and (y, z) values generated from them. For each PPRF key position, query Q times.
 - (b) Find $2^{\frac{2n}{\tau(1+\lceil \log \tau \rceil)}}$ number of PPRF key candidates such that

$$a^{(j)} = Z^{(j)}(\Delta_j^*) \oplus \vec{F}_y^{(j)}(\Delta_j^*) \cdot \vec{\mu} \oplus b\Delta_j^*.$$

Each candidate can be found in $O(D \cdot 2^{\frac{n}{1+\lceil \log(D-1) \rceil}})$ by Wagner's k -XOR algorithm for $k = D - 1$. Total time to find $2^{\frac{2n}{\tau(1+\lceil \log \tau \rceil)}}$ candidates costs

$$O(D \cdot 2^{\frac{n}{1+\lceil \log(D-1) \rceil}} \cdot 2^{\frac{2n}{\tau(1+\lceil \log \tau \rceil)(1+\lceil \log(D-1) \rceil)}}).$$

Q should be asymptotically this amount.

6. Among the candidates of τ -tuple of PPRF keys, find one such that

$$\vec{F}_x(\Delta^*) = \vec{s}^* \wedge \vec{F}_y(\Delta^*) = \vec{w}^*.$$

We note that the number of τ -tuples is $2^{\frac{2n}{1+\lceil \log \tau \rceil}}$. This problem can be solved by Wagner's k -XOR algorithm for $k = \tau$ on $\vec{F}_x(\Delta^*) \parallel \vec{F}_y(\Delta^*)$, which requires $O(\tau \cdot 2^{\frac{2n}{1+\lceil \log \tau \rceil}})$ time.

The total time cost is

$$O\left(2^{\lambda/2} + \tau D \cdot 2^{\frac{n}{1+\lceil \log(D-1) \rceil}} \cdot 2^{\frac{2n}{\tau(1+\lceil \log \tau \rceil)(1+\lceil \log(D-1) \rceil)}} + \tau \cdot 2^{\frac{2n}{1+\lceil \log \tau \rceil}}\right).$$

For all the parameter sets given in [16] which ranges from $(D, \tau) = (9, 15)$ to $(15, 9)$, the attack costs are similar and upper bounded by 2^{69} up to some constant multiple.

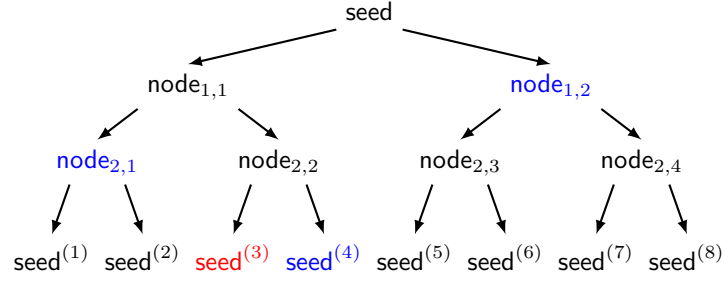


Fig. 2: Diagram of PPRF. In this example, $\text{seed}^{(3)}$ is the unopened node, and blue nodes are the PPRF key.

E Sign and Verify Algorithms of VOLEitH-SBC

The signing and verification algorithm for VOLEitH-SBC are presented in Figure 3 and Figure 4, respectively.

- **Inputs:** Secret key $\mathbf{sk} = (\vec{x}, \vec{y})$, public key $\mathbf{pk} = (\vec{u}, \vec{v})$ and a message $\mathbf{msg} \in \{0, 1\}^*$.
- 1. Sample $R_{\vec{x}} \leftarrow (\mathbb{F}_2)^{n-2}$ and a random seed for the cGGM trees derivation.
- 2. Compute Pre-Tree: $(\vec{x}^{[0]}, \dots, \vec{x}^{[2^\tau-1]}) \leftarrow \text{cGGM}(R_{\vec{x}}, R_{\vec{x}} \oplus \vec{x}, 2^\tau)$
- 3. Fold Pre-Tree: $(\vec{x}^{[0]_j}, \vec{x}^{[1]_j}) \leftarrow \text{Folding}(\vec{x}^{[0]}, \dots, \vec{x}^{[2^\tau-1]})$
- 4. For $j \in [\tau]$:
 - (a) Get Tree T_j : $(\vec{x}_j^{[0]}, \dots, \vec{x}_j^{[2^D-1]}) \leftarrow \text{cGGM}(\vec{x}^{[0]_j}, \vec{x}^{[1]_j}, 2^D)$
 - (b) Expand T_j and get $(\vec{y}_j^{[0]}, \dots, \vec{y}_j^{[2^D-1]})$ and $(z_j^{[0]}, \dots, z_j^{[2^D-1]})$
 - (c) Fold Tree T_j :

$$(\vec{x}_j^{[0]_i}, \vec{x}_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(\vec{x}_j^{[0]}, \dots, \vec{x}_j^{[2^D-1]})$$

$$(\vec{y}_j^{[0]_i}, \vec{y}_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(\vec{y}_j^{[0]}, \dots, \vec{y}_j^{[2^D-1]})$$

$$(z_j^{[0]_i}, z_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(z_j^{[0]}, \dots, z_j^{[2^D-1]})$$
 - (d) Let $z \leftarrow z_0^{[0]_0} \oplus z_0^{[1]_0}$
 - (e) Compute offsets $\delta_{\vec{y}_j} \leftarrow \vec{y} \oplus \vec{y}_j^{[0]_0} \oplus \vec{y}_j^{[1]_0}$ and $\delta_{z_j} \leftarrow z \oplus z_j^{[0]_0} \oplus z_j^{[1]_0}$
- 5. $h_0 \leftarrow \mathcal{H}(\mathbf{msg}, (\delta_{\vec{y}_j}, \delta_{z_j})_{j \in [\tau]})$
- 6. $\vec{\mu} \leftarrow \mathcal{H}(h_0)$
- 7. For $m \in [\tau \cdot D]$: $\alpha_m \leftarrow \mathcal{H}(m, h_0)$
- 8. Compute $\vec{A}_{F_x} \leftarrow \sum_{j=0}^{\tau-1} \sum_{i=jD}^{(j+1)D-1} x_j^{[0]_i} \alpha_i$
- 9. Compute $\vec{A}_{F_y} \leftarrow \sum_{j=0}^{\tau-1} \sum_{i=jD}^{(j+1)D-1} y_j^{[0]_i} \alpha_i$
- 10. Compute coefficients A and B as of Φ in (10).
- 11. Compute $b \leftarrow z + \vec{y} \cdot \vec{\mu}$
- 12. For $j \in [\tau]$:
 - (a) Compute $\vec{A}_{F_y}^{(j)} \leftarrow \sum_{i=jD}^{(j+1)D-1} y_j^{[0]_i} \alpha_i$ and $\vec{A}_{F_z}^{(j)} \leftarrow \sum_{i=jD}^{(j+1)D-1} z_j^{[0]_i} \alpha_i$
 - (b) Compute $a^{(j)} \leftarrow \vec{A}_{F_z}^{(j)} + \vec{A}_{F_y}^{(j)} \cdot \vec{\mu}$
- 13. $h_1 \leftarrow \mathcal{H}(h_0, \mathbf{msg}, A, B, (a^{(j)})_{j \in [\tau]}, b)$
- 14. For $j \in [\tau]$: $i_j^* \leftarrow \mathcal{H}(j, h_1)$
- 15. Compute the Pre-Challenge $i_p^* \leftarrow \sum_{j=0}^{\tau-1} \lfloor i_j^* / 2^{D-1} \rfloor \cdot 2^j$
- 16. Compute the Pre-Tree PPRF key $K_{i_p^*}$ and the PPRF key $K_{i_0^*, \dots, i_{\tau-1}^*}$
- 17. Output signature $\sigma \leftarrow (h_1, \text{seed}, (\delta_{\vec{y}_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]}, B, b, K_{i_p^*}, K_{i_0^*, \dots, i_{\tau-1}^*})$

Fig. 3: VOLEitH-SBC signature scheme - signing algorithm

- **Inputs:** Public key $\mathbf{pk} = (\vec{u}, \vec{v})$, a message $\mathbf{msg} \in \{0, 1\}^*$, and a signature $\sigma = (h_1, \mathbf{seed}, (\delta_{\vec{y}_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]}, B, b, K_{i_p}^*, K_{i_0}^*, \dots, K_{i_{\tau-1}}^*)$.
- 1. For $j \in [\tau]$: $i_j^* \leftarrow \mathcal{H}(j, h_1)$
- 2. Recompute and fold all trees (with one missing leaf each) using $K_{i_p}^*$, $K_{i_0}^*, \dots, K_{i_{\tau-1}}^*$, and $\mathbf{seed}$
- 3. For $(i, j) \in [D] \times [\tau]$, let $b_{i,j}$ denote the known position of the respective hypercube binary sharing
- 4. $\delta_{z_0} \leftarrow 0$
- 5. $h'_0 \leftarrow \mathcal{H}(\mathbf{msg}, (\delta_{\vec{y}_j}, \delta_{z_j})_{j \in [\tau]})$
- 6. $\vec{\mu} \leftarrow \mathcal{H}(h'_0)$
- 7. For $m \in [\tau \cdot D]$: $\alpha_m \leftarrow \mathcal{H}(m, h_0)$
- 8. For $j \in [\tau]$:
 - Compute $\Delta_j^* \leftarrow \sum_{i=jD}^{(j+1)D-1} b_{i,j} \alpha_i$
 - Compute $\vec{F}_y^{(j)}(\Delta_j^*) \leftarrow \sum_{i=jD}^{(j+1)D-1} \alpha_i (\vec{y}_j^{\llbracket b_{i,j} \rrbracket_i} + b_{i,j} \delta_{\vec{y}_j})$
 - Compute $Z^{(j)}(\Delta_j^*) \leftarrow \sum_{i=jD}^{(j+1)D-1} \alpha_i (z_j^{\llbracket b_{i,j} \rrbracket_i} + b_{i,j} \delta_{z_j})$
 - $a^{(j)'} \leftarrow Z^{(j)}(\Delta_j^*) \oplus \vec{F}_y^{(j)}(\Delta_j^*) \cdot \vec{\mu} \oplus b \Delta_j^*$
- 9. $\Delta^* \leftarrow \sum_{j=0}^{\tau-1} \Delta_j^*$
- 10. $\vec{F}_x(\Delta^*) \leftarrow \sum_{m=0}^{\tau \cdot D-1} x_m^{\llbracket b_m \rrbracket} \alpha_m$
- 11. $\vec{F}_y(\Delta^*) \leftarrow \sum_{m=0}^{\tau-1} \vec{F}_y^{(j)}(\Delta_j^*)$
- 12. Compute $\Phi(\Delta^*)$ as

$$\left(\vec{u} \cdot \begin{pmatrix} \vec{F}_x(\Delta^*) \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{F}_y(\Delta^*) \\ 0 \\ 1 \end{pmatrix} \right) - \left(\vec{u} \cdot \begin{pmatrix} \vec{F}_y(\Delta^*) \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{F}_x(\Delta^*) \\ 1 \\ 0 \end{pmatrix} \right)$$
- 13. $A' \leftarrow \Phi(\Delta^*) - B \Delta^*$
- 14. $h'_1 \leftarrow \mathcal{H}(h'_0, \mathbf{msg}, A', B, (a^{(j)'})_{j \in [\tau]}, b)$
- 15. If $h'_1 = h_1$ output **ACCEPT**, otherwise output **REJECT**

Fig. 4: VOLEitH-SBC signature scheme - verification algorithm